Singapore Institute of Technology
Information and Communications Technology Cluster

ICT4001 Capstone Project

Capstone Project Final Report Submission Form (Form E2)

A. Preamble

This form duly filled and signed must be attached to the Capstone Project Final Report submitted.

Candidate Particulars:

(Name of Candidate): Jevan Chow Jun Kiat

(Organization): Wizlynx Pte Ltd

(Project Period): 19/2/2024 to 15/11/2024

Main Academic Supervisor Details:

(Name): Spiridon Bakiras

(Email Address): spiridon.bakiras@singaporetech.edu.sg

(Contact Number): 6592 3324

(Designation): A/Prof

Industry Supervisor Details:

(Name): Lim Zhi Xun

(Email Address): zhixun.lim@wizlynxgroup.com

(Contact Number): 9387 2943

(Designation): Senior Information Security Consultant

(Department / Division): wizlynxgroup - Governance, Risk and Compliance

B. Declaration of Conformity

B.1 Declaration by Industry Supervisor

I hereby acknowledge that I have read and understood the contents of the Capstone Project Final Report, and deem the contents appropriate for release to the Singapore Institute of Technology for use in the assessment of the candidate.

Signature 

Name of Industry Supervisor: Lim Zhi Xun
Designation: Senior Information Security Consultant
Department: wizlynxgroup - Governance, Risk and Compliance
Contact No: 9387 2943
Email: zhixun.lim@wizlynxgroup.com
Date: 13/11/2024

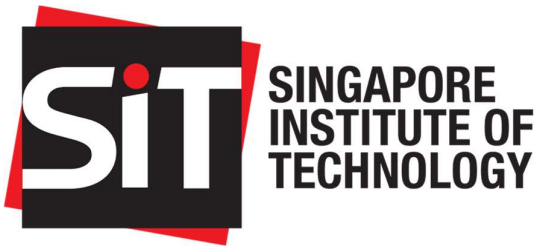
B.2 Declaration by Candidate

I hereby acknowledge that I have engaged and discussed with my Industry Supervisor on the contents of the Capstone Project Final Report, and have sought approval for the release of the report to the Singapore Institute of Technology.

Signature 

Name of Candidate: Jevan Chow Jun Kiat
Contact No: 8301 7702
Email: 2101144@sit.singaporetech.edu.sg
Date: 13/11/2024

END OF FORM E



Information and Communications Technology Cluster

VigilanceHub Training Platform

Capstone Project Final Report (Wizlynx Pte Ltd)

For the project period from 19/2/2024 to 15/11/2024

Jevan Chow Jun Kiat
2101144

Name of Industry Supervisor: Lim Zhi Xun

Name of Academic Supervisor: Spiridon Bakiras

Submitted as part of the requirement for ICT4001 Capstone Project

Table of Contents

Capstone Project Final Report for the Project VigilanceHub Training Platform, from 19/2/2024, to 15/11/2024

1. Introduction

In the realm of cybersecurity in the working environment, acquiring knowledge in this field is paramount. As technology is constantly evolving and is playing an integral role in organizations, so has the presence of cyber-attacks, with many reports of technology misuse ranging from phishing attempts to data breaches surfacing in recent years. While it may not be feasible to achieve fully attack-free computer systems for all employees in an organization, the next best thing would be for employees to play their part by understanding the many dangers of cyber-attacks and inculcating good awareness of cybersecurity concepts in the workplace, so that each and everyone will know how to react and respond to a potential attack.

This report aims to address the need for comprehensive cybersecurity awareness initiatives, doing so in the form of a cybersecurity awareness training platform as part of a capstone project. This report also acknowledges that the strength of an organization's cybersecurity posture lies not only in advanced technologies but, equally, in the vigilance and preparedness of its workforce. By understanding the evolving threat landscape and empowering employees with the knowledge to identify, prevent, and respond to potential threats, we can significantly enhance our organization's resilience against cyber-attacks.

2. Capstone Project Details

2.1. Project Overview and Inspiration

The capstone project involves creating a dynamic security awareness training platform aimed at fostering a proactive cybersecurity culture both within an organization and for individual users. This training platform is named VigilanceHub and is developed in the form of a web application (the terms ‘platform’, ‘application’ and ‘website’ are used interchangeably in this report).

My designation in my IWSP company is ‘Information Security Consultant – GRC’, and part of my job scope requires me to facilitate in conducts of security awareness trainings. Online platforms to conduct such awareness trainings exist too, and my IWSP company does utilise an existing platform for security awareness. However, the GRC team in my company has limited staff and with other departments only depending on the platform to learn various topics occasionally, some concerns can arise regarding other staff not remembering what they learnt about security awareness. Another gap identified is that such training platforms adopted by organizations require people to be actually employed by the organization or pay a subscription fee (for common users) to utilise these platforms. Therefore, my project aims to go one step further and incorporate valuable training topics that would be beneficial for all users, be it in the workplace or in everyday life.

By integrating various features such as engaging training topics and quizzes, the platform seeks to educate users on critical topics including phishing awareness, password security, and fundamentals for various areas in cybersecurity such as data protection and incident response. The platform prioritizes security, user-friendly interfaces, and scalability.

Continuous improvement is facilitated through research, user feedback via discussion forums, and occasional updates to training content, ensuring that the platform remains adaptable to evolving cybersecurity challenges. In the long run, the training materials may be updated, where applicable, to align with the latest happenings in the cybersecurity landscape. Ultimately, the goal is to empower individuals within the organization to become vigilant and informed contributors to overall cyber resilience.

2.2. Project Objectives

This project encompasses several objectives which are mainly aimed towards improving users' awareness in cybersecurity, and these objectives are more thoroughly explained below.

1. Educational Effectiveness
    - Create engaging and effective training modules that increase awareness and understanding of cybersecurity threats. These training materials have several videos embedded to aid with the normal training content, plus a few questions for users to think about when going through the topics. There are eight topics that I have decided for my training materials and a brief description of each is explained in Section 3.5.2.
    - Before unlocking the quiz materials, users will have to do a pre-training quiz consisting of 20 multiple-choice questions (MCQ). Their score will not affect the rest of the training materials as it is just to see where each user stands with regards to their knowledge in cybersecurity, so that they can have a clearer picture of which topics to put more focus on.
    - Upon completion of all training topics, a final quiz will unlock, consisting of 30 MCQs with questions from all eight topics. Users have unlimited attempts to take any of the quizzes if they wish to improve their score.

2. Integrating unique elements
    - As other such security awareness platforms do already exist, I aimed to make my project stand out more by incorporating more appropriate features for such a platform if possible. Such features include:
        a. Engaging training topics with videos links and rhetorical questions that draws users' attention.
        b. Feedback on quiz results
            i. Users can try out quizzes on the platform and their results will be shown upon completion of each quiz immediately.
            ii. Topic quizzes where a user scores below half are highlighted so that the user is aware of which areas they can improve on so that they can continue to revise on the topics before retaking the quiz.
        c. A discussion forum for users to share their thoughts on the platform such as help on the training topics or other cybersecurity-related matters.

3. Ease of accessibility
    - As of the submission date of this report, the source codes of VigilanceHub have been uploaded to GitHub. There is an executable (exe) file in the repository which serves as an alternate way to run the application. Users are required to follow the instructions stated in the README file in the repository before running the executable file to start the program, as errors in the program may arise if various packages or configurations are missing. The link to the GitHub

repository for VigilanceHub can be found in Section 5 and in the appendix of this report.

2.3. Industry Relevancy

This project is relevant to the industry in terms of the training materials which are the backbone of this project. As I am undergoing my IWSP as an Information Security Consultant in the GRC department, part of my job scope involves cybersecurity awareness and ensuring that my company's clients and employees within my company adopt a cybersecurity mentality in both the workplace and in everyday life. My project focuses on security awareness training, which is something that employees of my organization should be familiar with. Listed below are some reasons why my project is relevant to the industry.

1. Enhancing security posture
    - VigilanceHub equips users with the knowledge and skills needed to recognize and respond to cybersecurity threats. This, in turn, enhances the overall security posture of users and clients.

2. Meeting compliance requirements
    - Many industries follow specific compliance standards and regulations related to cybersecurity training. Implementing a robust training platform can help users and clients working in these organizations by providing tips and guidelines that they can use to meet these requirements, ensuring they remain compliant with industry standards.

3. Reduce human-related security risks
    - A considerable number of cybersecurity incidents stem from human error. By providing effective security awareness training, the project helps to educate users on various fundamentals and cyber-attacks. One aim of the platform is to mitigate human-related risks, reducing the likelihood of incidents such as phishing attacks and social engineering exploits.

4. Cater to individual's needs
    - There is no requirement for users to read all the materials, as the platform is structured in such a way that users can read any contents as and when they wish; whichever content caters to their cybersecurity needs.
    - However, before any of the training materials can be read, users are required to do a 20-question pre-training quiz first. This is to roughly gauge where the user stands on their prior cybersecurity knowledge. Finally, if they want to take the final quiz to test their knowledge on all the topics taught, they then are required to go through all training materials and their respective quizzes.

5. Displays commitment by the industry
    - The use of VigilanceHub can be a unique selling point for a cybersecurity service provider, showing that the industry is committed to providing their utmost service. This will in turn, attract potential clients.

2.4. Project Scope and Complexity

The project scope involves the development of a security awareness training platform to educate and empower users within an organization on cybersecurity threats, best practices, and compliance standards. The platform will integrate cybersecurity topics, quizzes, and regular updates all in one website, to enhance user engagement and effectiveness. Last

but not least, this project as a whole, aims to foster a security-aware culture.

In terms of complexity, the platform will be developed in the form of a website running locally on the user's system, incorporating HTML, CSS and JavaScript elements. While emphasizing a user-friendly interface, security features, and scalability, the platform includes a login/logout function where users input their email as one of their identification requirements.

Implementing secure features within this web application is also important while challenging, as the functionality of a website is dependent on how secure its features are. Components such as session/cookie management and proper password requirements must be considered, as without them, the platform can be easily intercepted by malicious actors.

2.5. Project Outputs and Deliverables

The project output includes the development of a comprehensive security awareness training platform. This platform will feature interactive training modules, personalized learning paths, and regular updates aimed at enhancing user engagement and knowledge retention. The outputs also comprise a user-friendly interface, robust security measures, and scalability features to accommodate the organization's evolving needs.

The project deliverables consist of tangible assets and documentation essential for the successful implementation of this platform. These deliverables encompass training modules which are easy to learn, user guides, and documentation detailing the platform's features and functionalities. Continuous improvement plans, regular updates, and adaptive features will serve as ongoing deliverables, ensuring the longevity and effectiveness of the platform in addressing the organization's (or even individual's) cybersecurity training needs.

3. VigilanceHub Training Platform Guide

This section documents the features and flow of the VigilanceHub training platform that the student has developed as part of his Capstone Project while undergoing his Integrated Work-Degree Study Programme (IWSP) at Wizlynx Pte Ltd. A complete flowchart of the platform is shown in Figure 1 for better clarity.

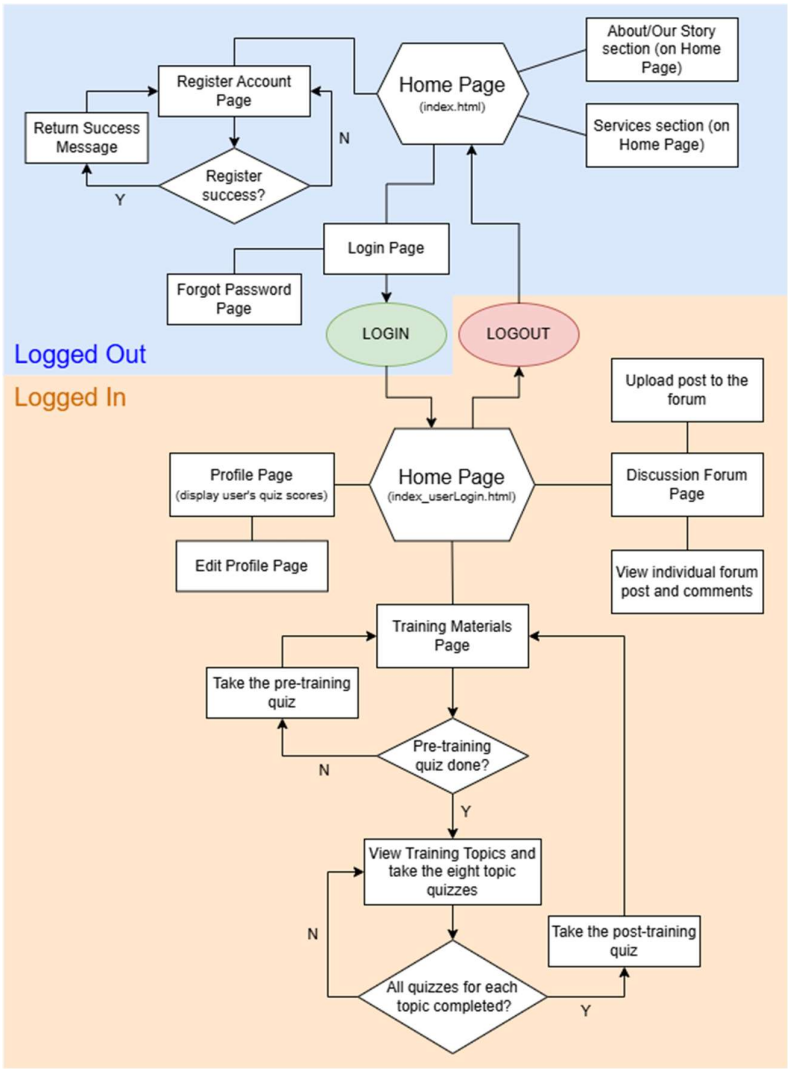


Figure 1: Flowchart of VigilanceHub

The flowchart is divided into two sections, 'Logged Out' and 'Logged In' which are coloured in blue and orange respectively. There are four pages in the 'Logged Out' section, namely the Home Page (**index.html**), Register Account Page, Login Page and Forgot Password Page (which is accessible through the Login Page). To be able to access the pages in the 'Logged In' section, a user must first register an account via the Register Account Page. Upon success, the user can then log in via the Login Page and access all the pages in the 'Logged In' section, starting with the logged in Home Page (**index_userLogin.html**).

In the 'Logged In' section, the user can visit their Profile Page (and edit their user details if they wish) or the Discussion Forum. For the main selling point of VigilanceHub, the user may access the training menu from the Training Materials Page, where they are first required to complete a pre-training quiz to be allowed to access the main training slides where the content is located.

3.1. Home Page

Upon launching VigilanceHub, the user will be directed to the home page, or **index.html**. This home page will have three sections, namely the main section (the content seen at the

top of the page), ‘About’ section, and finally the ‘Services’ section. A brief description of each section is mentioned below.

- Main Section
  - This section displays an image slideshow relating to cybersecurity, to give the feel of a website on cybersecurity to users who access this page. A button with the text ‘Join Us!’ is included below this slideshow to encourage new users to register an account.
- About Section
  - This section gives a brief description of the VigilanceHub training platform, explaining what this platform is about and the inspiration behind its fruition.
- Services Section
  - This section explains the features that VigilanceHub offers for users, and they include:
    - Robust training materials
    - Quizzes to test users’ knowledge
    - Tracking of users’ scores for continuous improvement
    - Discussion forum for users
  - Below this section, there is a short explanation on why users should register with VigilanceHub, affirming the services provided in this platform which will convince users to adopt VigilanceHub to improve their cybersecurity knowledge.

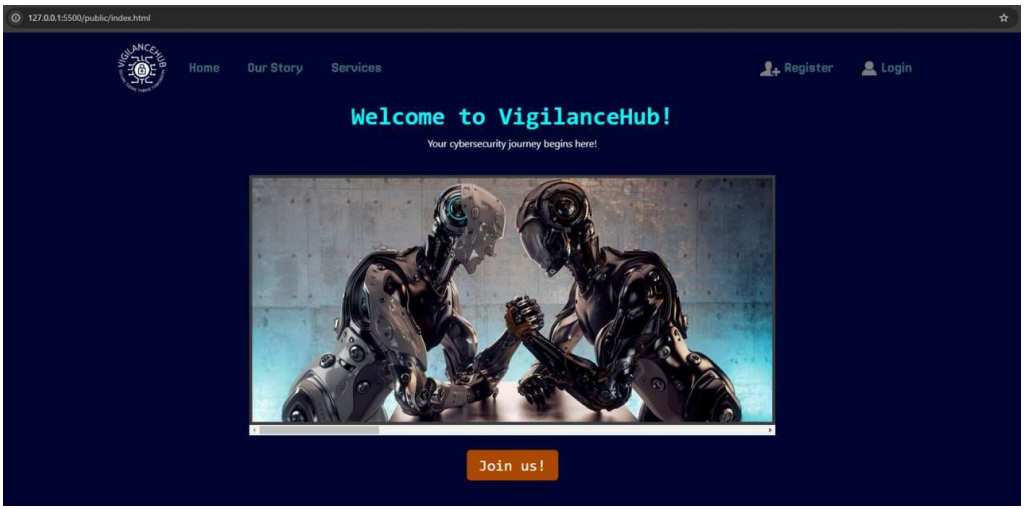


Figure 2: index.html (Main section)

All the three sections are part of **index.html** instead of their own HTML page. This is to make use of the navigation bar (navbar) such that clicking the respective link on the navbar will scroll the page down to the respective section. The navbar code is placed in a separate HTML file called **navbar_home.html**, with JavaScript functions in each respective HTML code/page to load the correct navbar. For example, all logged-out/non-authenticated pages will load **navbar_home.html** and logged-in/authenticated pages will display a different navbar. Figure 3 shows a close-up look of the navbar in all the logged-out pages.

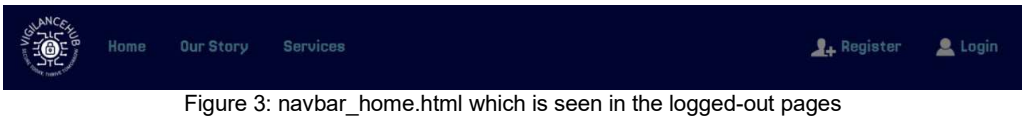


Figure 3: navbar_home.html which is seen in the logged-out pages

3.2. Register and Login Functions

3.2.1. Register Account

Clicking ‘Register’ in the navigation bar will direct the user to the Register Account page, or **register.html**. Over here, the user is required to enter a username, email and password. Following common password guidelines, the password must comprise of at least eight characters, including one uppercase and lowercase letter, digit and special character.

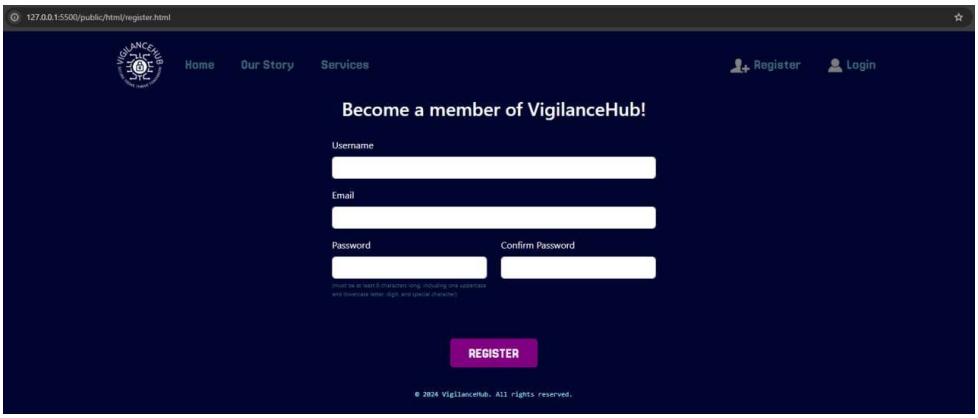


Figure 4: register.html

If the password does not meet the requirements or the value in the ‘Confirm Password’ field does not match the value in the ‘Password’ field, an error message will display depending on the condition that was not fulfilled. If all conditions are fulfilled, a message will display below the ‘Password’ and ‘Confirm Password’ field, telling the user that a confirmation email has been sent to the email address given in this page. Upon clicking the link in the confirmation email, the user will be successfully registered and can now log in to their account via the login page, or **login.html**. A more technical explanation of the email confirmation process is found in Section 3.8.3 of this report.

3.2.2. Login User

Clicking ‘Login’ in the navigation bar will direct the user to the Login page, or **login.html**. In this page, there are two text fields, labelled ‘Email or Username’ and ‘Password’. In the first field, inputting either the user’s username or email address is accepted as a login credential, while the second field requires the user’s password corresponding to the username/email registered to the account.

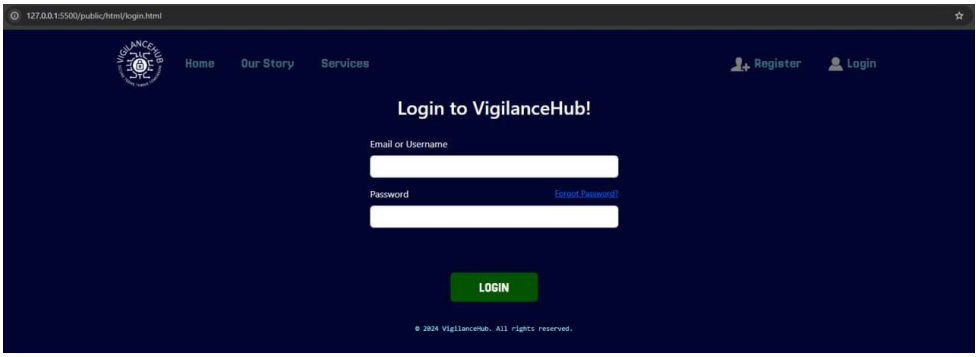


Figure 5: login.html

If either the email/username or password does not match an entry in the database, an error message stating 'Incorrect username/email or password' will display, and the user must try again. An important point to note is that the password in the database has been hashed using the **bcrypt** algorithm for security purposes, which will be explained in further detail in Section 3.8.2 of this report.

If a user forgets their password, they can click the 'Forgot Password?' link just above the password field and they will be directed to a simple-looking HTML page called **forgot_password.html**. In here, they will be asked to input their email address registered with their account, after which, an email with a link to reset their password will be sent to their email address. The reset password function will be explained in further detail in Section 3.8.6 of this report.

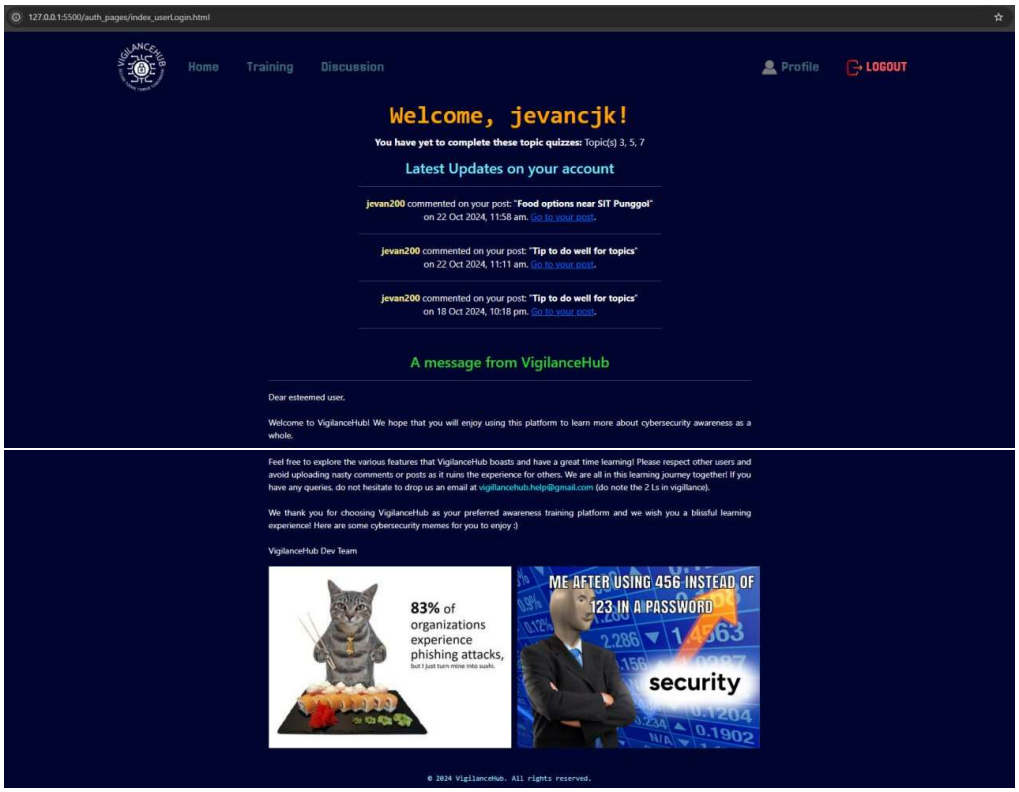
3.3. Home Page after Login

To access nearly all the features of VigilanceHub, users are required to log in to their account. Upon successful login of the user, they will be redirected to the home page after logging in, or **index_userLogin.html**. Logged in (authenticated) pages use another navbar called **navbar_userLogin.html** as this navbar includes links to pages that would only be accessible via authenticating or logging in. Figure 6 shows the look of the navbar for logged-in pages.

Figure 6: navbar_userLogin.html which is seen in the logged-in pages

The 'Training' link in the navbar will bring users to the training menu and the quizzes which will be explained more in detail in Section 3.5, while the 'Discussion' link will bring users to a discussion forum where various users can create posts and comment on existing posts. This will be explained more in detail in Section 3.6.

Back to **index_userLogin.html**, at the top of the page, a text in the middle will display "Welcome, <username>", to indicate that that specific user has successfully logged in, and the <username> corresponds to the username of the logged in user, regardless of whether they logged in via username or email. Below this text, another text will display indicating the topic quizzes that the user has yet to attempt (i.e. get a score for that quiz). Therefore, if a user registers an account and logs in for the first time, the message will indicate all eight quizzes, which in some kind of way, prompts the user to try out the training topics and quizzes first. In the same HTML page, a section to display latest updates to the user's account is also included. This section caters to the aforementioned discussion forum feature where any comments posted on a post made by the logged-in user will be updated here. Figures 7a and 7b show the look of **index_userLogin.html** for the user 'jevancjk', where this user has attempted a few quizzes and received comments from other users on a few posts.



Figures 7a and 7b: index_userLogin.html

3.4. Profile and Edit Profile Page

In the right-hand side of **navbar_userLogin.html**, users can click the 'Profile' link to access their profile page, or **profile.html**. In here, they will see their username, avatar (if they wish to use one) and recent quiz scores, with a text at the bottom indicating the training topics recommended to revise on based on the user's scores. If the user has not attempted any of the topic quizzes or has achieved a score below half of the maximum score (10) in any of them, the topic number will be included in this message. Refer to Sections 3.5.2 and 3.5.3 for more detailed information regarding the training topics and quizzes. Figure 8 shows the look of the profile page for the user 'jevancjk'.

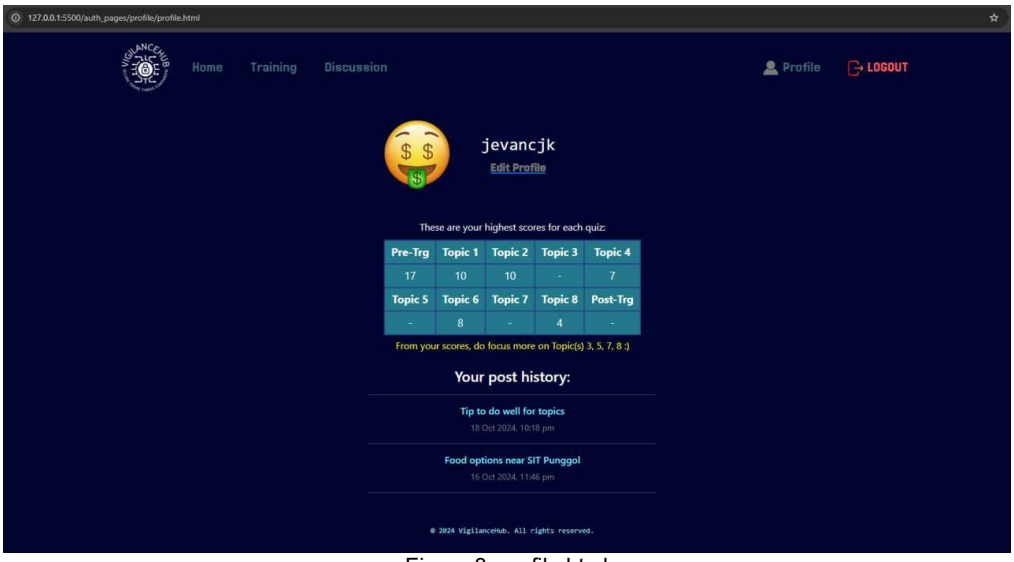


Figure 8: profile.html

A feature included in **profile.html** is a history of the posts that the user has uploaded to the discussion forum, with their most recent post displayed at the top. Users can click the post title of any of their posts and will be redirected to **forum_post.html** where they can view more details about their post and any comments made by other users. The discussion forum feature will be explained more in detail in Section 3.6.

Finally, there is an 'Edit Profile' button in **profile.html** which will redirect the user to **edit_profile.html**. Before the user can edit their credentials, they are required to input their current password for verification. The rest of the elements in this HTML page will remain hidden until the correct password is entered. In **edit_profile.html**, the user can change their username or password and can also assign an avatar from an existing selection to display on their profile if they wish.

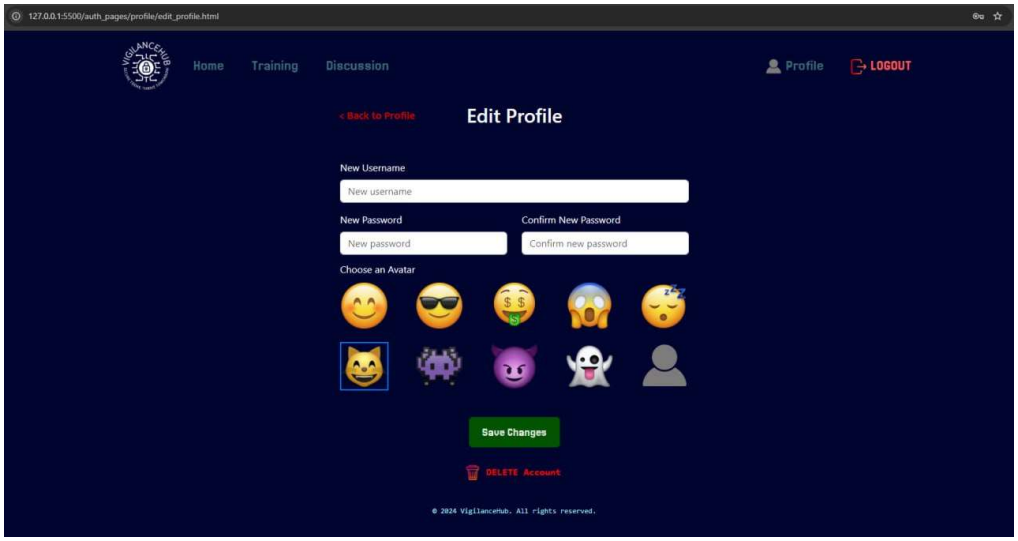


Figure 9: edit_profile.html

Upon successfully implementing any changes and clicking the 'Save Changes' button, a message will pop up on the screen telling the user to log out of VigilanceHub and re-login.

This is to ensure that a new session can be created with the newly updated user's credentials, which are the userID and username. Should the user not re-login, some of the content seen in the HTML pages may be inaccurate such as displaying of the username in certain pages.

Below the 'Save Changes' button, there is an option for the user to delete their account if they wish. Clicking the link will prompt the user asking if they are sure that they would like to delete the account, and that doing so is an irreversible action. Confirming this prompt would complete the action with an alert confirming the deletion of the account, and the user will be redirected to **index.html**. The functionality of the account deletion is explained in Section 3.8.5.

3.5. Training Materials

The VigilanceHub training platform would not be complete without proper training materials on cybersecurity, and proper designing of the webpages was done to ensure that users are intrigued in the training materials that this training platform has to offer.

3.5.1. Main Menu

After the user logs in, they can click on the 'Training' link on the left of the navigation bar, and they will be redirected to the main menu of the training materials page, or **training_menu.html**. Figure 10 shows the design of **training_menu.html**.

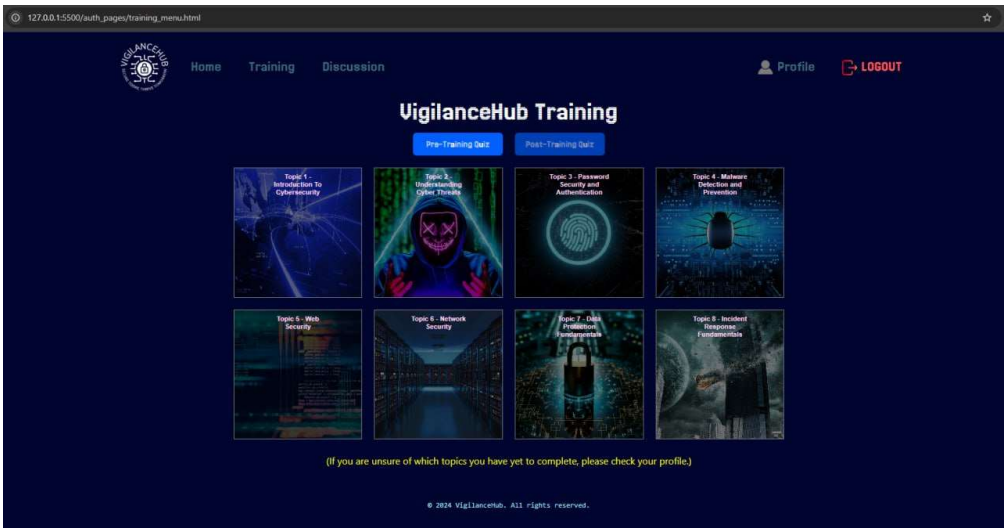


Figure 10: training_menu.html

From the training menu, there are various options that a user can do. They can do the Pre-Training Quiz or Post-Training Quiz, which are the two blue buttons at the top of the page, right below the navbar. If they want to access the training topics instead, they can click any of the eight topics as seen in the middle of the page, and the respective topic will display in another page.

To encourage progressive learning for newly registered users, the **training_menu.html** page is configured such a way that, when a user visits this page for the first time, a message will pop up prompting the user to take the pre-training quiz first. All training materials and the post-training quiz buttons are disabled on the first visit and will remain

that way until the user attempts the pre-training quiz and gets a score, leaving the pre-training quiz button the only available option as of now. Their score of the pre-training quiz has no effect on the rest of the flow of the program as the sole purpose of the pre-training quiz is simply to see where the user stands on their amount of prior knowledge on cybersecurity so that they can boost their knowledge more by learning from the training topics that VigilanceHub offers.

Regardless of their Pre-training quiz score, the eight training topics will unlock and the user is free to study any of the topics as they wish, in any order. However, the post-training quiz button will remain disabled until the user attempts and gets a score on each of the eight topic quizzes.

3.5.2. Training Topics

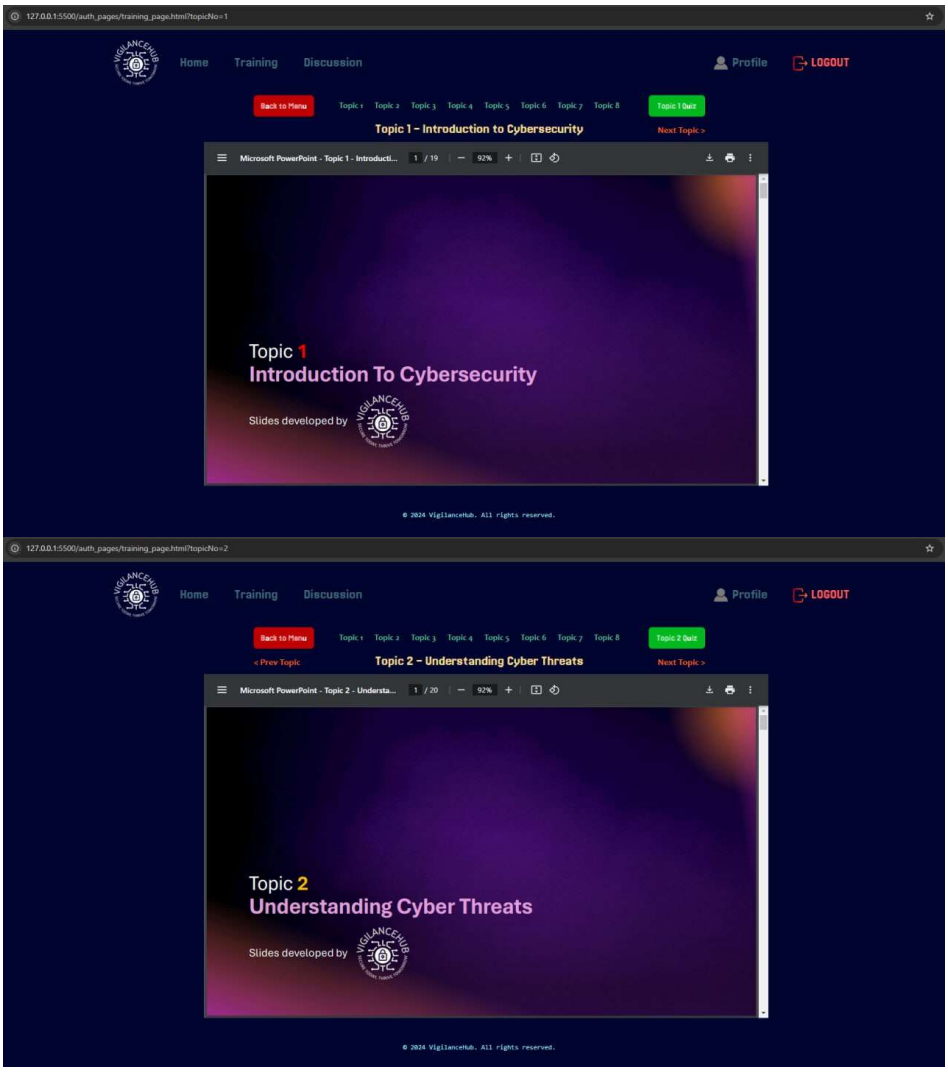
VigilanceHub comprises eight training topics covering various areas in cybersecurity that is beneficial for users to learn about. Table 1 shows the list of topics included, and the overall content that a user would expect to be covered per topic.

Topic No.	Topic Name	Content Covered
1	Introduction to Cybersecurity	- Overview of cybersecurity fundamentals - Common terms used in cybersecurity - CIA Triad and AAA Model - Access control measures
2	Understanding Cyber Threats	- Common cyber threats - Importance of phishing awareness - Mitigating cyber threats
3	Password Security and Authentication	- Best practices for creating strong passwords using NIST password guidelines - Multi and Two-Factor Authentication (Something you know/have/are)
4	Malware Detection and Prevention	- Common types of malwares - Detecting malware - Malware prevention tips
5	Web Security	- Components of a URL - HTTP methods and CRUD - Port numbers - HTTP status codes
6	Network Security	- IP and MAC addresses - OSI Model - Network components - Network monitoring practices
7	Data Protection Fundamentals	- Importance of data protection - Data protection-related legislations - Data Loss Prevention - Documents and terms used in data protection
8	Incident Response Fundamentals	- Importance of incident response procedures - NIST Incident Response Life Cycle - Common response plans used

Table 1: VigilanceHub topic list

Clicking on any of the eight topics in the main menu brings the user to another HTML page, named **training_page.html**. In this page, there is a PDF window with the training slides

for each topic, similar to how it is done on the Xsite Learning Management System (LMS). To differentiate the same HTML page displaying different topics, the source code was configured in JavaScript to dynamically display the corresponding topic to the parameter value in the URL. An explanation of the configuration logic can be found in Section 3.9.1. Figures 11a and 11b show the rough look of **training_page.html**, where the parameter corresponds to **Topic 1** and **Topic 2** respectively as an example. The remaining six topics have similar configurations with their own respective topic slides in the PDF window.



Figures 11a and 11b: Topics 1 and 2 as seen in training_page.html

As seen in the above figures, users can either go back to the main menu via the red 'Back to Menu' button, go any other topic via the teal-coloured links, or click on the green 'Topic X Quiz' to access the 10 questions for that specific topic. They can also go to the next or previous topic in sequence by clicking the orange links right below the red and green buttons. The window is styled to fit the content nicely where the browser's (Google Chrome is the recommended browser for VigilanceHub) zoom percentage is 100%, with dimensions of 1000 by 610 pixels (pixels is the default measure unit for dimensions in HTML if it is not mentioned in the code) tested on a standard 16:9 aspect ratio. Figures 12a and 12b show the HTML code to style the PDF window to the correct dimensions and show the correct PDF slides.

```html
<div class="container text-center training-container mt-2">
    <button id="menuButton" class="btn btn-danger mb-3 menu-button">Back to Menu</button>
    <h3 id="topicTitle" class="text-center" style="font-family: 'Jersey 20'; font-size: 25px; color: #FFE296;"></h3>
    <div class="prev-topic">
        <a id="prevTopic" href="#"><h5>< Prev Topic</h5></a>
    </div>
    <div class="next-topic">
        <a id="nextTopic" href="#"><h5>Next Topic ></h5></a>
    </div>
    <button id="topicQuizButton" class="btn btn-danger mb-3 topic-quiz-button"></button>
    <object id="topicPdf" class="pdf" width="1000" height="610"></object>
</div>
```

<…rest of code…>

```javascript
const topics = [
    { num: 1, title: "Introduction to Cybersecurity", pdf: "topics/Topic 1 - Introduction to Cybersecurity.pdf" },
    { num: 2, title: "Understanding Cyber Threats", pdf: "topics/Topic 2 - Understanding Cyber Threats.pdf" },
    { num: 3, title: "Password Security and Authentication", pdf: "topics/Topic 3 - Password Security and Authentication.pdf" },
    { num: 4, title: "Malware Detection and Prevention", pdf: "topics/Topic 4 - Malware Detection and Prevention.pdf" },
    { num: 5, title: "Web Security", pdf: "topics/Topic 5 - Web Security.pdf" },
    { num: 6, title: "Network Security", pdf: "topics/Topic 6 - Network Security.pdf" },
    { num: 7, title: "Data Protection Fundamentals", pdf: "topics/Topic 7 - Data Protection Fundamentals.pdf" },
    { num: 8, title: "Incident Response Fundamentals", pdf: "topics/Topic 8 - Incident Response Fundamentals.pdf" }
];

const topic = topics.find(t => t.num == topicNo);

if (topic) {
    $("#topicTitle").text(`Topic ${topic.num} - ${topic.title}`);
    $("#topicPdf").attr("data", topic.pdf);
    $("#topicQuizButton").text(`Topic ${topic.num} Quiz`);

    $("#nextTopic").attr("href", `training_page.html?topicNo=${topic.num + 1}`);
    $("#prevTopic").attr("href", `training_page.html?topicNo=${topic.num - 1}`);

    $("#title").text(`Topic ${topic.num}`);

    if (topic.num == 1) {
        $("#prevTopic").hide();
    }

    if (topic.num == 8) {
        $("#nextTopic").hide();
    }

    // Load the correct quiz on quiz_page.html
    $("#topicQuizButton").on('click', function() {
        window.location.href = `http://127.0.0.1:5500/auth_pages/quiz/quiz_page.html?quiz=topic${topic.num}`;
    });
}
```

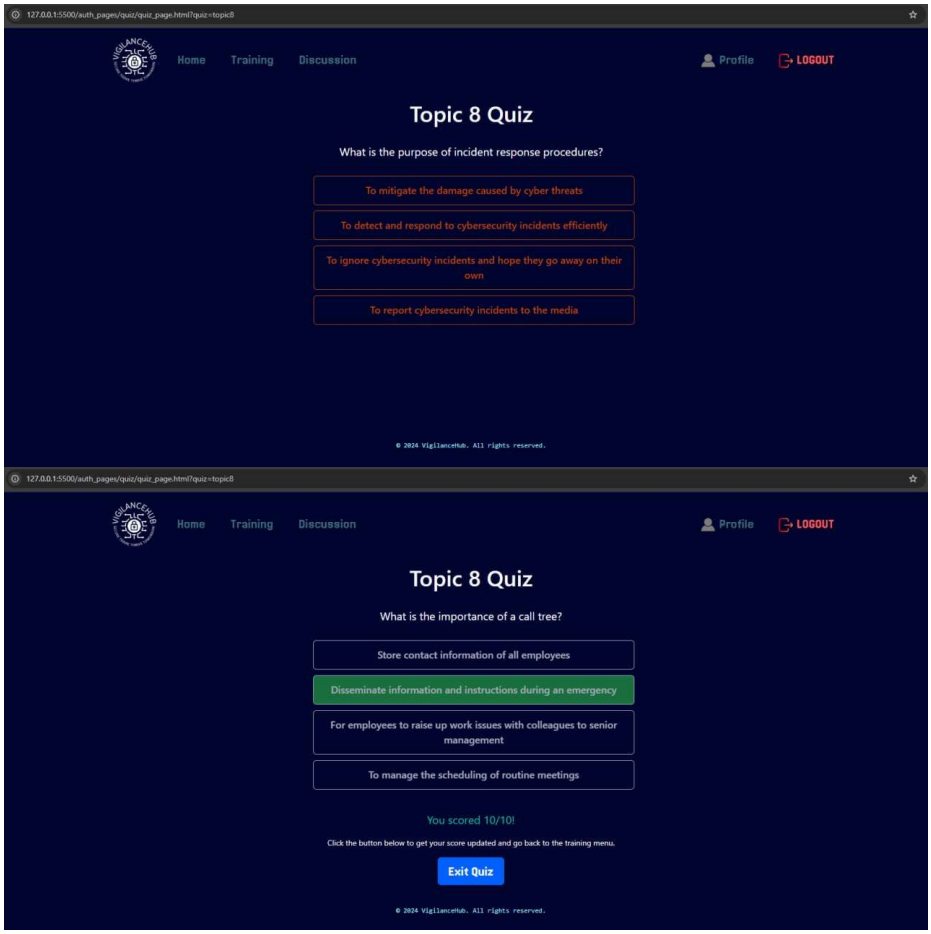
Figures 12a and 12b: HTML code to display the respective PDF windows

3.5.3. Quizzes

There are ten quizzes that a registered user can take across the training platform. They include the pre-training quiz, one mini quiz for each of the eight topics and a post-training quiz. Instead of storing the questions and answers in the MySQL database, CSV files [18] were utilised, with one file for each of the ten quizzes. In each CSV file, there are six values for each row of data, where each of the ten quizzes follows the same format. They six values are, in order: question, optionA, optionB, optionC, optionD, correctAnswer.

Users can access each topic quiz by clicking the respective ‘Topic X Quiz’ button on **training_page.html**. This will bring the user to the quiz page for **quiz_page.html**. The questions from the matching quiz CSV file will display one question at a time, whereby the user must click the option which they feel is correct. Doing this will reveal if the user picked the correct or incorrect answer, where the correct answer will display in green while the incorrect answer will display in red, and after a short while, the next question will display. This process continues for all questions in the quiz until the final question, where the user’s score and a button to go back to the training menu will be displayed. Clicking this button will redirect the user back to **training_menu.html**, while on the backend, the database will update the specific quiz score for that specific user in the **users** table. Figure 13a displays the look of **quiz_page.html** for a chosen quiz, Topic 8 for example, while Figure 13b shows

the quiz result message and button shown at the end of the Topic 8 quiz.



Figures 13a and 13b: quiz_page.html for Topic 8 quiz and the HTML after completing the quiz

3.6. Discussion Forum

Another beneficial feature that VigilanceHub offers is a discussion forum where users can create posts about things related to the training platform, such as the training topics and quizzes or other cybersecurity-related matters in general. Users can also comment on other users' posts to assist with their queries or simply giving some input. Clicking the 'Discussion' link in **navbar_userLogin.html** will redirect the user to the forum menu, or **forum.html**.

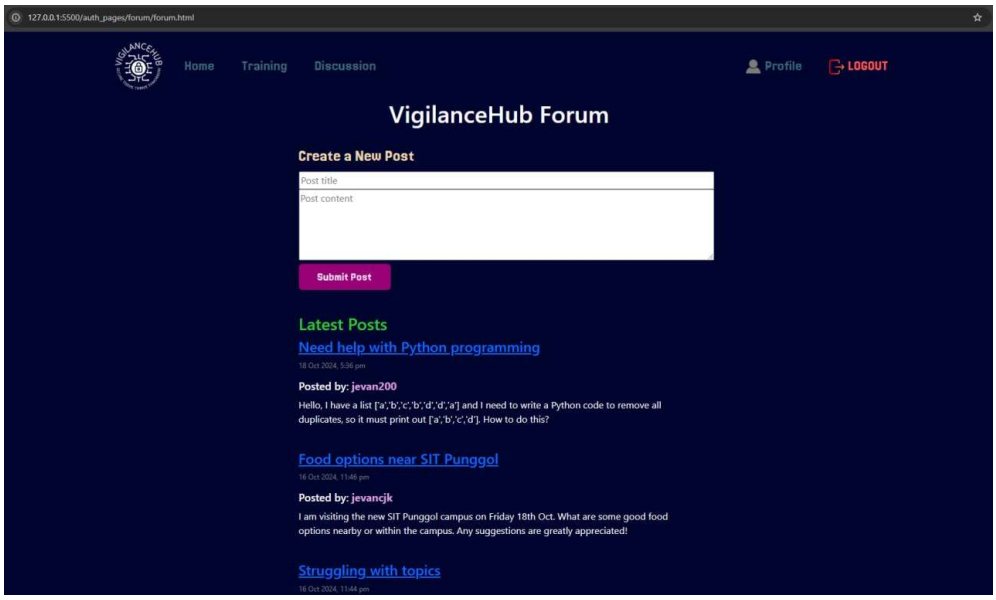


Figure 14: forum.html

At the top of **forum.html**, there are two text fields labelled ‘Post title’ and ‘Post content’ and a submit button for users to upload posts to the forum. A forum post consists of two components, ‘Title’ and ‘Content’. Therefore, a user is required to provide both a title and the main content of the post to successfully upload to the forum.

Below the title and content fields, users can see various posts uploaded by users, with the username of the uploader included. These posts are sorted by the most recent post at the top The title of each forum post is a link that can be clicked. This will redirect the user to another HTML page, called **forum_post.html**. In this page, the user can view more details about the clicked post, such as the title, uploader and the comment(s) from different users (if any) replying to the post. Figure 15 shows the look of **forum_post.html** with the topic of the post regarding ‘Food options near SIT Punggol’.

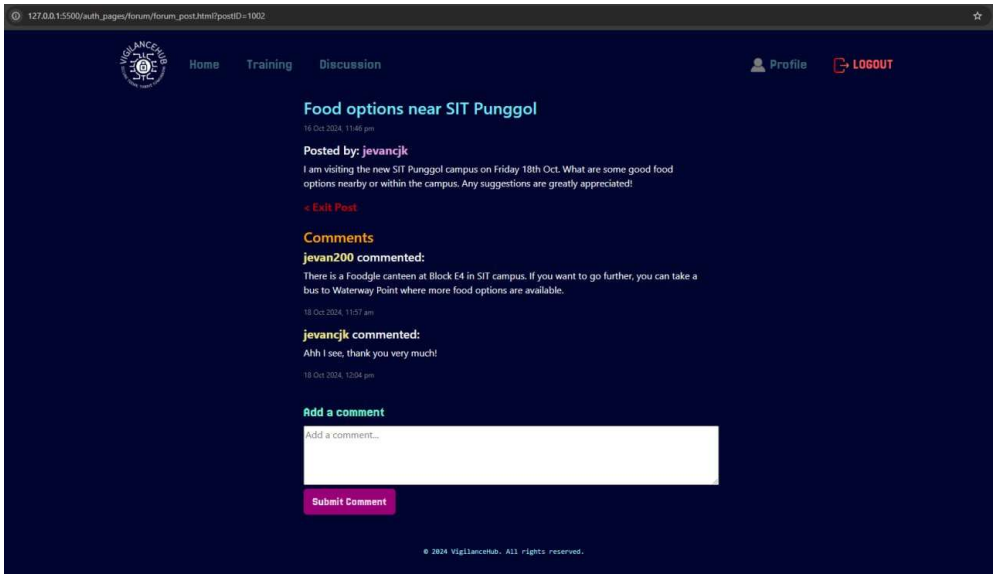


Figure 15: forum_post.html

3.7. SQL Database

Relevant data in the VigilanceHub training platform is stored in a SQL database, which was created on MySQL Workbench. There are three tables that store necessary data, and they are named **users**, **posts** and **comments**. The MySQL connection is named 'VigilanceHub' while the schema where the three tables is stored is named 'vigilancehub'. Table 2 displays the three tables and the required columns for each table.

Table name	Columns	Data Type
users	userID	INT **(AI, PK)**
	username	VARCHAR(45)
	email	VARCHAR(255)
	password	VARCHAR(255)
	avatar	VARCHAR(255)
	emailConfirmed	TINYINT(1)
	confirmationToken	VARCHAR(255)
	tokenExpiry	DATETIME
	pre_training	TINYINT
	topic1 – topic8 (8 columns)	TINYINT
	post_training	TINYINT
posts	postID	INT **(AI, PK)**
	userID	INT (FK to **users** table)
	username	VARCHAR(45)
	postTitle	VARCHAR(255)
	postText	TEXT
	postDate	TIMESTAMP
comments	commentID	INT **(AI, PK)**
	postID	INT (FK to **posts** table)
	userID	INT (FK to **users** table)
	username	VARCHAR(45)
	commentText	TEXT
	commentDate	TIMESTAMP

Table 2: List of tables in MySQL database

In the INT data types for the userID, postID and commentID columns, AI stands for Auto-Increment, PK stands for Primary Key, and FK stands for Foreign Key.

In the users table, the userID, username, email and password are stored here. The pre_training, topic1-8 and post_training columns refer to the individual quiz scores that each user gets after doing a specific quiz. The TINYINT data type stores integer values from 0 to 255. Therefore, using TINYINT for the pre-training, eight topics and post-training scores is a reasonable choice as compared to INT as the maximum score for any quiz is 30 (post-training).

For the posts and comments table, each post and comment is assigned a postID and commentID respectively, similar to how users are assigned a userID. Foreign keys are adopted in these tables with the following explanations.

- In the posts table, **userID** is a foreign key to the users table which identifies the uploader of the post.
- In the comments table, **postID** is a foreign key to the posts table which identifies

the post that the comment is replying to or commenting on. On the other hand, **userID** is a foreign key to the users table which identifies the user who commented on the post.

3.8. Secure Features

Being a website running on a local server, secure features were implemented to enhance security throughout the VigilanceHub training platform to prevent unauthorized access and ensure that user data remains protected within the web application, maintaining the overall system integrity.

3.8.1. Port Configurations

Running an application on a localhost requires one or more ports to be utilised and properly configured, and the VigilanceHub training platform is no different. Proper port configuration ensures that the various parts of the application can interact securely and efficiently, minimizing the risk of conflicts and vulnerabilities. Table 3 below lists the port numbers utilised in VigilanceHub.

Port No.	Purpose
587	Encrypted communication via SMTP using Nodemailer
3000	Node.js and Express.js which handles all the backend logic on the server-side
5500	Go-Live extension which hosts the local server where users view the UI of the webpage

Table 3: List of port numbers used and their purpose

Another port used in this project but is not mentioned in the codes is Port 3306, which points to MySQL database connections by default. As the default port was used in this project, specifying it explicitly in the **app.js** configuration is not necessary as the MySQL client already instructs port 3306 to be used unless otherwise specified.

3.8.2. Password Hashing using bcrypt

After registering an account with VigilanceHub, the user's password is hashed using the bcrypt hashing algorithm with a cost factor of 10, meaning that the algorithm will will go through 2^{10} or 1024 iterations to slow down the hashing process. This ensures that even if an attacker obtains the hashed passwords, significant computational resources is required to obtain the original passwords, thus enhancing security against brute-force attacks. Figure 16 shows a snippet of the code in the '/register' backend route in **app.js** that implements bcrypt to hash the passwords.

```
app.post('/register', async (req, res) => {
    const { username, email, password } = req.body;
    try {
        const hashedPassword = await bcrypt.hash(password, 10);
        const sql = "INSERT INTO users (username, email, password) VALUES (?, ?, ?)";
        pool.query(sql, [username, email, hashedPassword], (err, results) => {
            if (err) {
                console.error(err);
                res.status(500).json({ success: false, message: 'Failed to register. Database error' });
                return;
            }
            res.json({ success: true, message: 'Account created successfully! Please login.' });
        });
    } catch (error) {
        console.error(error);
        res.status(500).json({ success: false, message: 'Failed to register. Server error' });
    }
});
```

Figure 16: Snippet of code in app.js indicating the use of bcrypt

When viewing the password column in the database, the password for each user follows the format **'$2b$10$.....'** whereby **$2b** indicates the usage of the OpenBSD version of the bcrypt algorithm, while **10$** refers to the cost factor, which is 10 in this case. Figure 17 shows the list of users in the MySQL database with the values in the password column hashed using bcrypt. Do note that the exact same password was registered for all four users seen in the database, and the password hashing courtesy of bcrypt ensures that even the exact same passwords do not return the same values after hashing.

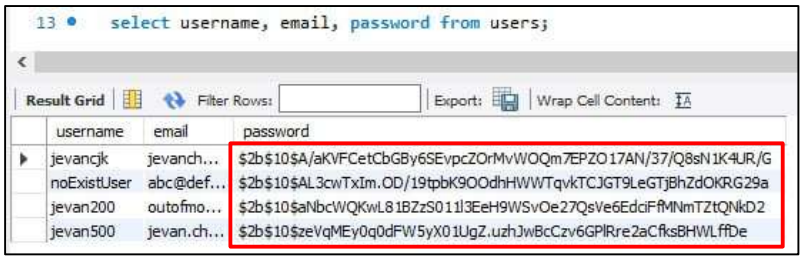


Figure 17: Table of users in the MySQL database

3.8.3. Email confirmation upon registering

When users input all their details in **register.html** and click the 'Register' button, a message will appear below the Password and Confirm Password text fields, indicating that their credentials are valid, and an email has been sent to the user's email address requiring them to confirm their registration. The database for the users table will update with a new row for the user with their username, email and password stored. The *emailConfirmed* column will be set to a value of 0, equivalent to the value of False in Boolean as MySQL does not recognize Boolean syntax but rather, the TINYINT(1) data type (the (1) in TINYINT(1) refers to the display width of the TINYINT data type and valid values range from -128 to 127. It can be used in Boolean context where the value of 0 equates to False while the value of 1 equates to True).

Simultaneously, a 20-byte confirmation token is generated using the **crypto** module in Node.js, with the link in the email pointing to **http://127.0.0.1:3000/confirm/${token}**, where the {token} path matches the 20-byte token to ensure that the link is unique only to one user. 20 bytes were considered for this project instead of the conventional 16 as, while 16 bytes (128 bits) is generally considered secure, 20 bytes (160 bits) increases the maximum number of potential token values, making brute-force or collision attacks even less likely to occur. Figures 18a and 18b show the implementation of the **crypto** module to generate a unique confirmation link in the /register route and the /confirm/:token path respectively, as configured in **app.js**.

```
const token = crypto.randomBytes(20).toString('hex');
const confirmationLink = `http://127.0.0.1:3000/confirm/${token}`;
```

```
// if valid token, delete value of confirmation token and token expiry date
const updateSql = "UPDATE users SET emailConfirmed = 1, confirmationToken = NULL, tokenExpiry = NULL WHERE confirmationToken = ?";
pool.query(updateSql, [token], (err) => {
    if (err) {
        console.error(err);
        res.status(500).send('Error confirming registration.');
        return;
    }
    setTimeout(() => {
        res.redirect('http://127.0.0.1:5500/misc_html/registration_success.html');
    }, 1500);
});
```

Figures 18a and 18b: Implementation of crypto module and code to handle email confirmation

Upon further inspection of Figure 18b, it is noted that the *emailConfirmed* value will only be set to 1 **after** the user clicks on a valid email link. Therefore, if a user tries to login to their account from **login.html** without confirming their email, an error message will display, telling them to do so. This also doubles up as an email validation feature, prompting users to input their actual email address instead of a non-existent one, to ensure that users can only utilise the platform proper with a legitimate email address.

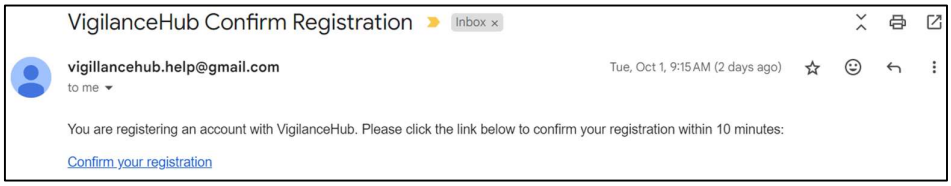


Figure 19: Automated email sent to a valid email address

Upon clicking the link in the email, an interim HTML page called **registration_success.html** will open (redirected from the original URL pointing to port 3000), indicating that the user has successfully registered an account. Clicking the button below will redirect the user to **login.html** where they can now log in, and in the MySQL database, the *emailConfirmed* value will be set to 1. Simultaneously, the *confirmationToken* and *tokenExpiry* values will be removed (set to NULL) as the user would be successfully registered and there is no need to keep these values. This also prevents SQL injection attacks as an attacker would be unable to exploit these two (now non-existent) values for malicious purposes by inserting malicious SQL code to bypass the registration process.

Finally, VigilanceHub implements an email expiry feature, where a time limit of 10 minutes [7] is automatically set upon clicking the 'Register' button in **register.html**, provided that the user's credentials are valid and that a confirmation email has been sent. This means that the user has 10 minutes to click the link provided in the email. Failure to do so will result in the email link redirecting the user to another interim HTML page called **registration_expired.html** if the user clicks the link after the 10 minutes is up. This HTML page includes a message indicating that the email link has expired, prompting the user to re-register, with a button below that redirects users to **register.html**. Simultaneously, the MySQL database will delete the row of the user that was affected by the email expiry once they click the expired link which opens up **registration_expired.html**, to confirm that that user's data no longer needs to be stored in the database. Figures 20a and 20b show the codes to set the 10-minute confirmation token validity and the handling of expired email tokens being clicked.

```
const tokenExpiry = new Date();
tokenExpiry.setMinutes(tokenExpiry.getMinutes() + 10);
```

```
const currentDate = new Date();
if (currentDate > new Date(user.tokenExpiry)) {
    // if token expired, delete row from database
    pool.query("DELETE FROM users WHERE confirmationToken = ?", [token], (err) => {
        if (err) {
            console.error('Error deleting expired token:', err);
            res.status(500).send('Error processing expired token.');
            return;
        }
        setTimeout(() => {
            res.redirect('http://127.0.0.1:5500/misc_html/registration_expired.html');
        }, 1500);
    });
    return;
}
```

Figures 20a and 20b: 10-minute token validity set and code to handle expired confirmation token

3.8.4. Session Management and Cookies

Session management and cookies [10, 11] were implemented in VigilanceHub to prevent unauthorized access to pages which require authentication. As seen in the flowchart back in Section 3, all pages that the user can only access upon logging in are considered 'logged in' or authenticated pages, and they fall under a folder named *auth_pages* in the main project directory of VigilanceHub. The step-by-step process of this implementation is as follows:

1. When a user logs in via the /login route, their session data such as their userID and username is stored on the server.
2. A session cookie is sent to the browser, which is named 'login_session'. This cookie is assigned a unique value which identifies the user for future requests (if a new session is created, the value changes as it is unique only to one particular session).
3. The session cookie is included with every request the user makes to the server, allowing the server to retrieve the session data and verify the user's identity.

Figure 21 shows the session cookie name and ID when the 'Cookies' section is selected under *Inspect* > *Application* > *Cookies* > *http://127.0.0.1:5500* in Google Chrome, where the cookie name is 'login_session' and the ID/value refers to the long string of characters where in this example or particular session, starts with 's%3AYkg4….'.

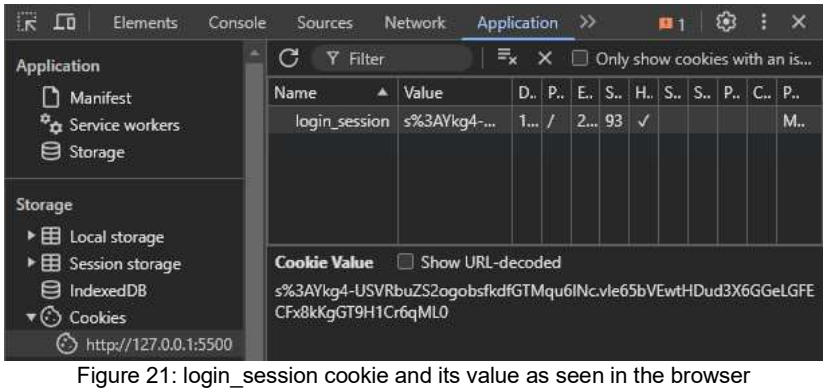


Figure 21: login_session cookie and its value as seen in the browser

The /checkAuth route is used to detect if a user has been authenticated via logging in. If a user has successfully logged in with their credentials, the authenticated JSON value will change to true or 1 (it is 0 or false by default).

If the user logs out of their account, the cookie will be removed, and the user will be unable

to re-access the web pages under 'auth_pages' unless they re-login which will generate a new cookie value while retaining the name of 'login_session'. Likewise, if a logged-in user manually right clicks and deletes the cookie like the one in Figure 21, their authenticated status will toggle to **false**, and the user is unable to access the authenticated pages and must similarly re-login. A test case on cookie deletion is demonstrated in Section 4.2.

The deletion of the cookie also takes place when a user deletes their account through **edit_profile.html**. As a user has to be logged in to access **edit_profile.html**, a session cookie similar to the one in Figure 21 will be active and can be seen in the 'Application' tab under 'Inspect'. Once the user successfully deletes their account, the session will end and similar to the logout process, the cookie will be removed, and the user cannot re-access their now deleted account even with a newly generated cookie value.

3.8.5. Account Deletion

As mentioned towards the end of Section 3.4, VigilanceHub offers an option for users to delete their account [19] if they wish, with the link found in **edit_profile.html**. Clicking the link triggers a prompt confirming with the user if they are certain that they would like their account to be deleted. Clicking 'Cancel' will close the prompt and the user can continue accessing VigilanceHub's features with their account intact, while clicking 'OK' confirms their account deletion and they would be logged out and redirected to the homepage, or **index.html**.

In the backend, the system first deletes associated comments and posts from the database, which are linked to the user's account via foreign key constraints. This can be seen in the **app.js** code where three const variables are declared in order:

1. *deleteUserComments*: Deletes all comments made by the user on posts in the forum, including their own posts
2. *deleteUserPosts*: Deletes all posts uploaded by the user on the forum
3. *deleteUser*: Deletes the user and all its associated data (username, password etc.)

Due to the foreign key constraints, data in the **comments** table must be deleted first, followed by data in the **posts** table and finally, from the **users** table itself. This order of deletion is necessary to maintain database integrity since direct deletion of the user would violate these constraints and the database will return an error. By wrapping the entire process in one 'transaction', the system ensures that all related data (comments, posts, and the user record) is either completely deleted or none at all, avoiding partial deletion scenarios. Additionally, the session is securely destroyed similar to a typical logout process, and the authentication cookie is cleared, preventing unauthorized access to the system post-deletion.

```
// Get a connection from pool
pool.getConnection((err, connection) => {
    if (err) {
        console.error('Error getting connection from pool:', err);
        return res.status(500).json({ success: false, message: 'Failed to get database connection' });
    }

    connection.beginTransaction((err) => {
        if (err) {
            console.error('Error starting transaction:', err);
            connection.release(); // after each query, release connection to avoid connection leaks
            return res.status(500).json({ success: false, message: 'Failed to start transaction' });
        }

        const deleteUserComments = 'DELETE FROM comments WHERE userID = ?';
        connection.query(deleteUserComments, [userID], (err, result) => {
            if (err) {
                return connection.rollback(() => {
                    connection.release();
                    console.error('Error deleting comments:', err);
                    return res.status(500).json({ success: false, message: 'Failed to delete comments' });
                });
            }

            const deleteUserPosts = 'DELETE FROM posts WHERE userID = ?';
            connection.query(deleteUserPosts, [userID], (err, result) => {
                if (err) {
                    return connection.rollback(() => {
                        connection.release();
                        console.error('Error deleting posts:', err);
                        return res.status(500).json({ success: false, message: 'Failed to delete posts' });
                    });
                }

                const deleteUser = 'DELETE FROM users WHERE userID = ?';
                connection.query(deleteUser, [userID], (err, result) => {
                    if (err) {
                        return connection.rollback(() => {
                            connection.release();
                            console.error('Error deleting user:', err);
                            return res.status(500).json({ success: false, message: 'Failed to delete user' });
                        });
                    }

                    connection.commit((err) => {
                        if (err) {
                            return connection.rollback(() => {
                                connection.release();
                                console.error('Error committing transaction:', err);
                                return res.status(500).json({ success: false, message: 'Failed to complete transaction' });
                            });
                        }

                        // destroy session upon successful deletion of user
                        req.session.destroy((err) => {
                            if (err) {
                                connection.release();
                                console.error('Error destroying session:', err);
                                return res.status(500).json({ success: false, message: 'Failed to log out' });
                            }

                            // delete cookie ID and value
                            res.clearCookie('login_session');
                            connection.release();
                            return res.json({ success: true, message: 'Account deleted successfully' });
                        });
                    });
                });
            });
        });
    });
});
```

Figures 22a and 22b: Delete account transaction code in app.js

3.8.6. Forgot/Reset Password

To log in to their account, a user's username/email and password are required. If a user forgets their password [20], they will need to click the 'Forgot Password?' link in **login.html** and input their email address in the interim **forgot_password.html**. Confirming their email will result in a unique link being sent to the inputted email address and clicking the email link will redirect the user to another interim HTML page called **reset_password.html**.

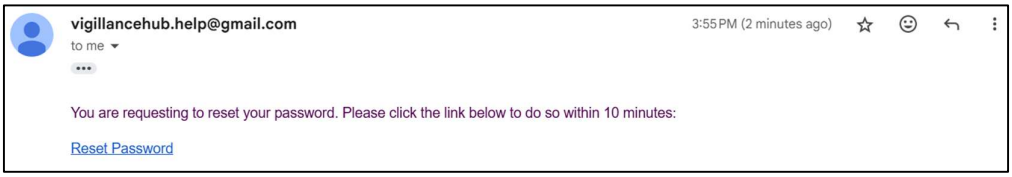
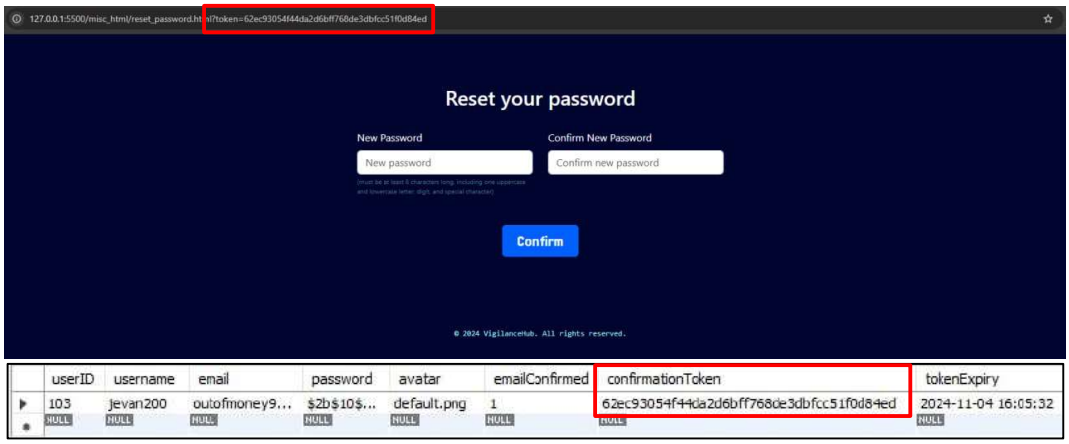


Figure 23: Automated email with a link to reset password

Implementing similar logic to the email confirmation link when registering, the password reset link is secured with a unique confirmation token. Therefore, the reset password feature utilises the confirmationToken and tokenExpiry columns in the users table in the database as well to enhance efficiency. When a user clicks the email link, the backend generates a 20-byte secure token using the **crypto** module in Node.js and just like the email confirmation token, the password reset token is valid for 10 minutes and is updated to the tokenExpiry column in the database.

Back to the frontend, when the user gets directed to **reset_password.html**, they are required to input a new password and confirm it. Just like a new user entering their password when registering, the newly-reset password must comprise of at least eight characters, including one uppercase and lowercase letter, digit and special character, and an error message will display if any requirements were not met.



Figures 24a and 24b: reset_password.html and the corresponding confirmationToken value in the database

If the user exceeds the 10-minute timeframe between the receiving of the email and the confirming of their new password in **reset_password.html**, a separate error message will display, informing the user that the link has expired and is prompted to get a new password reset email from the 'Forgot Password?' link in **login.html**, with a button leading to said page displaying below the password fields. If the user resets their password in time, the new password will be securely hashed with **bcrypt** and the database is updated with the new hashed password, with a message informing the user of the successful password reset and that they can log in with the new password.

An important point to note is that, elsewhere in the database, the confirmationToken and tokenExpiry fields are cleared regardless of whether the user had successfully reset their password or not. This is especially important for the latter scenario, where a user may not want to reset their password immediately after failing the first time, therefore it is vital to ensure that any secure tokens are discarded once they are no longer needed by any functions. These measures ensure that expired or malicious tokens cannot be reused which prevents SQL injection attacks on the database.

3.9. Other technical features

3.9.1. Dynamic Content Rendering with URL components

To reduce the number of unnecessary HTML pages used in VigilanceHub, URL components were implemented in some of the HTML pages to divide the specific page into different sections instead of creating a separate HTML file for that specific section. The URL components utilised are Parameters and Anchor, and the implementation details are listed below.

1. Parameters
    - URL parameters are denoted by a '?' prefix, followed by key-value pairs that provide additional information or specify certain behaviours on a webpage. Firstly, clicking on any of the eight topics in the training menu brings the user to another HTML page, named **training_page.html**. On this page, there is a PDF window with the training slides for each topic, similar to how it is done on the Xsite Learning Management System (LMS). To differentiate the same HTML page displaying different topics, the source code was configured in JavaScript to dynamically adjust the content displayed based on the topic number passed through the URL parameter 'topicNo', ranging from 1 to 8 to correspond with the eight topics of VigilanceHub. When **training_page.html** is loaded, the *getQueryParam* JavaScript function retrieves the 'topicNo' parameter from the URL. If the 'topicNo' parameter is not present, the page defaults to Topic 1, updating the URL accordingly. Figure 25 shows a snippet of the code where parameters are implemented, enclosed in a <script> tag in **training_page.html**.

```
if (topic) {
    $("#topicTitle").text(`Topic ${topic.num} - ${topic.title}`);
    $("#topicPdf").attr("data", topic.pdf);
    $("#topicQuizButton").text(`Topic ${topic.num} Quiz`);

    $("#nextTopic").attr("href", `training_page.html?topicNo=${topic.num + 1}`);
    $("#prevTopic").attr("href", `training_page.html?topicNo=${topic.num - 1}`);

    $("#title").text(`Topic ${topic.num}`);

    if (topic.num == 1) {
        $("#prevTopic").hide();
    }

    if (topic.num == 8) {
        $("#nextTopic").hide();
    }

    // Load the correct quiz on quiz_page.html
    $("#topicQuizButton").on('click', function() {
        window.location.href = `http://127.0.0.1:5500/auth_pages/quiz/quiz_page.html?quiz=topic${topic.num}`;
    });
}
```

Figure 25: Snippet of code in training_page.html implementing parameters

In addition to the eight training topics, the ten quizzes are also configured with parameters in **quiz_page.html**, with the quiz name, such as topic1 or post_training can be seen in the URL as a parameter. The URL with parameters for the quiz name can also be seen in Figures 13a and 13b in Section 3.5.3.

- Another feature in VigilanceHub where parameters are used is in the discussion forum, where each post in the forum is assigned an ID, or postID in the database. Clicking on a specific post in **forum.html** will redirect the user to **forum_post.html**, where the content loaded on this page corresponds to the postID of the post that was made.

2. Anchor
    - URL anchors are denoted by a '#' prefix, used to scroll or load a specific section

of a web page, especially long ones or single-page applications where users need quick access to specific sections without the need to load a new page. When the user clicks on a link containing an anchor, the browser interprets the # followed by an identifier as a directive to navigate directly to the section of the page with the corresponding ID attribute. The anchor implementation can be seen in a snippet of the **navbar_home.html** code in Figure 26 below.

```
<ul class="navbar-nav ml-auto">
    <li class="nav-item-left">
        <a class="nav-link" href="index_home.html">Home</a>
    </li>
    <li class="nav-item-left">
        <a class="nav-link" href="index_home.html#about">Our Story</a>
    </li>
    <li class="nav-item-left">
        <a class="nav-link" href="index_home.html#services">Services</a>
    </li>
</ul>
```

Figure 26: Snippet of code from navbar_home.html

4. Manual Testing

In a website, regular testing is required during the development stage to ensure that the website produces the correct results based on various given inputs. This section documents the results of several manual tests performed in different parts of the VigilanceHub training platform.

4.1. Account registration

** Refer to Section 3.8.3 for a more detailed explanation.*

When all registration requirements are successful in **register.html**, a message will display on the HTML page, indicating that an email has been sent, prompting the user to check their email account given during registration and click the confirmation link provided within 10 minutes to be successfully registered with VigilanceHub. Excluding a successful registration, there are two possible scenarios that could arise from trying to register an account and they will be demonstrated in Sections 4.1.1 and 4.1.2.

4.1.1. Non-existing email address submitted

Users can input any valid email address with the correct syntax (e.g. *apple@pear.com*). However, as the user is required to click the link in the email given, submitting a non-existent email address will render the user unable to click on any link given to fully confirm their registration, as they will never be able to access this 'non-existing' email.

For testing of this component, a user was registered with a username of 'noExistUser' email address of *abc@def.com* at 5:43pm or 1743 hours. Clicking the 'Register' button still outputs the correct message on **register.html**, however, on the sender's end, a warning will be shown, indicating that the email message was not sent as the domain name could not be found.

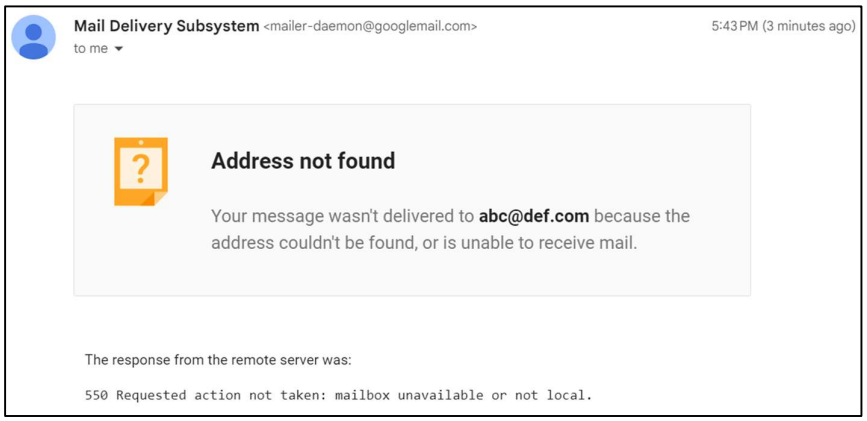


Figure 27: Sender's reply of sending to non-existent email

In the database, an entry will still be created for the user 'noExistUser'. However, the *emailConfirmed* value will stay at 0 and is unable to be set to 1 due to the absence of the email and link provided. From here, noExistUser tries to bypass the email confirmation and tries to log in immediately from **login.html**. However, inputting the correct credentials generated a message which prompted them to check their 'non-existent' email, and the user is unable to log in to view the features available.

Figure 28: Message on login.html prompting the user to confirm their registration

Figure 28 shows the data of the user noExistUser with the non-existent email of abc@def.com (the userID of 126 was assigned at the time of registration), leaving them no way of clicking a valid confirmation link to confirm their registration to allow access to other features which otherwise, require authentication. Do note the emailConfirmed value of 0 and tokenExpiry value of 1753 hours, which is 10 minutes from the time that the user was registered.

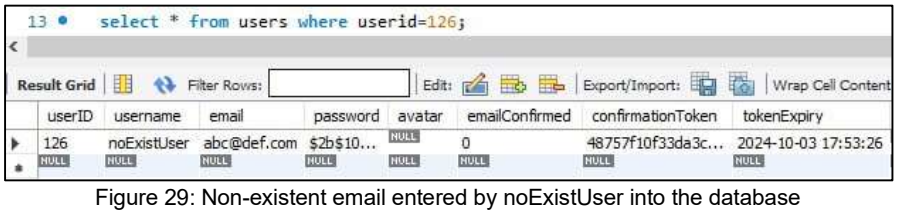
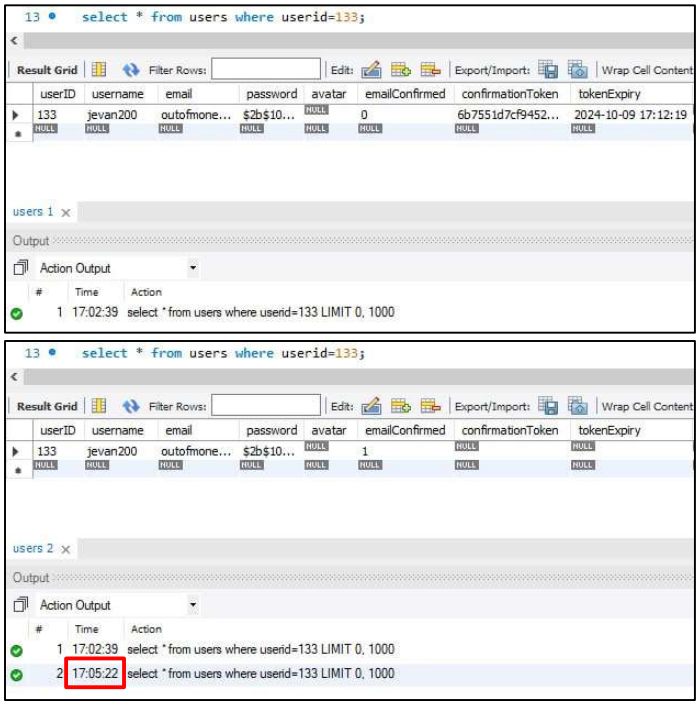


Figure 29: Non-existent email entered by noExistUser into the database

4.1.2. Clicking on an expired email confirmation link

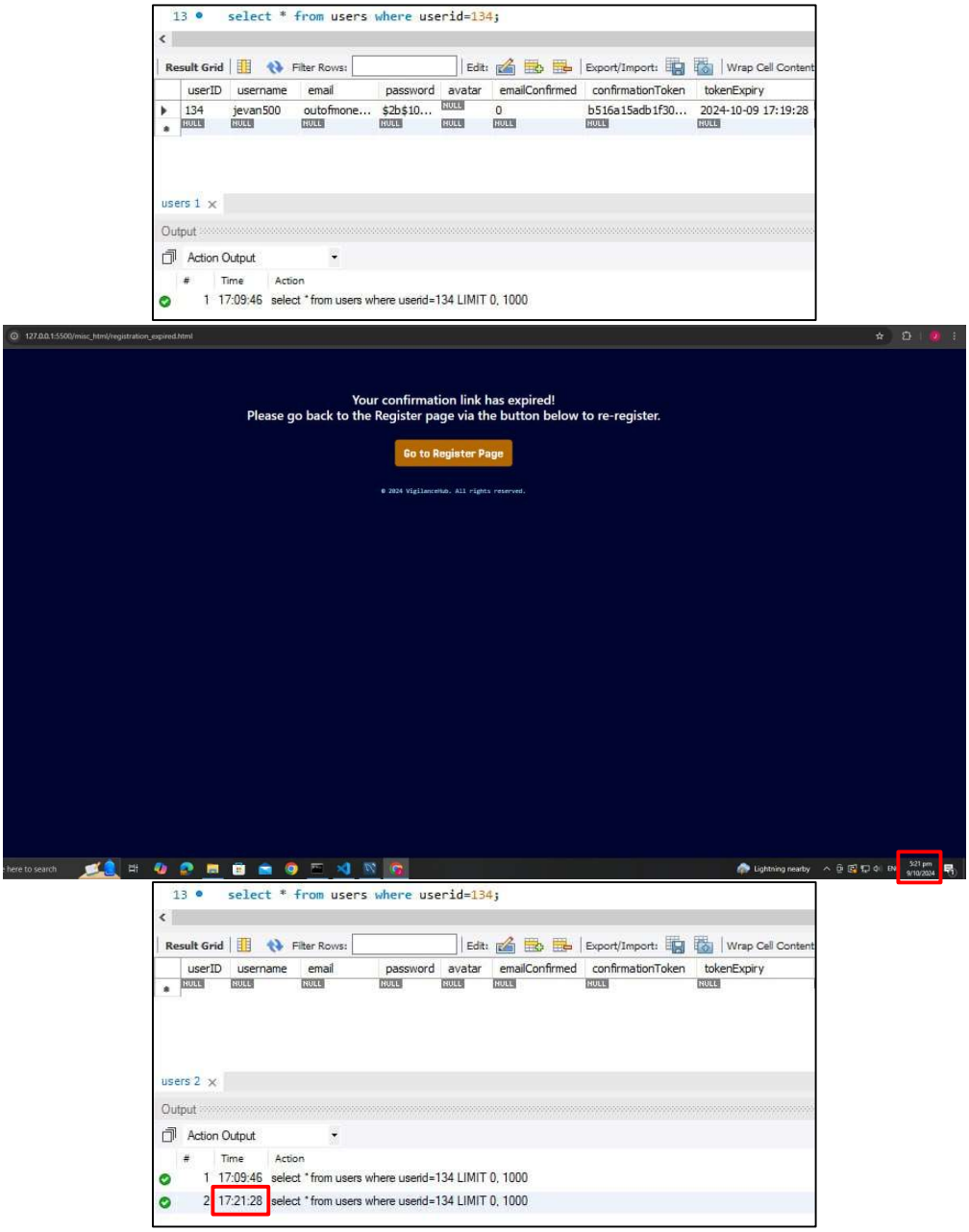
Upon clicking the 'Register' button and seeing the message about the email being sent, the confirmation link included in the email will be valid for 10 minutes. To test this out, two scenarios were executed.

In the first scenario, a user with the username 'jevan200' was registered with valid credentials, including a valid email address, and was assigned the userID of 133 in the database. The time when the user clicked 'Register' on **register.html** was detected to be 1702 hours, making the expiry 10 minutes later at 1712 hours. jevan200 clicks the email confirmation link within the stipulated time limit of 10 minutes, and **registration_success.html** is opened, allowing jevan200 to login and access VigilanceHub's features. Figures 30a and 30b display the data for jevan200 in MySQL Workbench before and after clicking the email confirmation link respectively. Do note the captured time at which the SELECT statements were executed as Figure 30b was taken at 1705 hours which occurred after jevan200 had clicked the email confirmation link.



Figures 30a and 30b: Data for user jevan200 before and after the link was clicked within 10 minutes

Moving on to Scenario 2, another user called 'jevan500' registers with valid credentials at 1709 hours (userID of 134 was assigned) and receives the same email as what jevan200 received. However, jevan500 exceeded the 10-minute timeframe and only opened the email confirmation at 1721 hours. Therefore, the **registration_expired.html** page opened instead, prompting jevan500 to re-register an account. On the backend MySQL database, the entry for jevan500 gets deleted, allowing the username of 'jevan500' to be reused for a new registration.



Figures 31a, 31b and 31c: Data for user jevan500 before clicking the link, registration_expired.html seen at 1721 hours and the deleted row when searching for userID=134

4.1.3 Clicking an email link multiple times

From Sections 4.1.1 and 4.1.2, users would either be notified of their successful registration to VigilanceHub upon clicking the email link within 10 minutes, or an expired registration if the 10-minute window exceeded. However, the email will still remain in the user's inbox with the link being able to be clicked again.

To test this out, user jevan200 leaves his desk for a toilet break without locking his computer and his email account left open. Unbeknownst to him, his colleague attempts to play a prank by locating the email sent by VigilanceHub and clicking the email link even

after jevan200 had already clicked it. However, clicking the link resulted in the webpage displaying a text saying, 'Invalid or expired token'. This is because the 20-byte confirmation token was deleted from the database when jevan200 clicked the email link for the first time, and trying to re-open the link with a (now) non-existent confirmation token resulted in the database unable to locate the corresponding token value and thus, determined the value from the email as invalid.

An important point to note is that this feature also applies to expired links that have been clicked before, as the user who exceeded the 10 minutes and clicked the link would have been redirected to the interim **registration_expired.html** page, and the entire row of data for that user would be deleted, including the confirmation token and its expiry time.

Figure 32: Invalid result after re-opening an email link

4.2. Removing of Session Cookie while session is active

** Refer to Section 3.8.4 for a more detailed explanation.*

When a user logs in to VigilanceHub, the session cookie with the name of 'login_session' is active and a unique string of characters is generated for the session's value. This cookie allows the user to access pages which require authentication within the platform. To test this out, user jevan200 logs in to VigilanceHub as per normal. By default, jevan200 is allowed to access all web pages that require authentication with the session cookie. Upon clicking 'Inspect' and navigating to 'Application', jevan200 is able to see the session cookie and its details. jevan200 then right-clicks on the session cookie and clicks 'Delete', removing the cookie entirely.

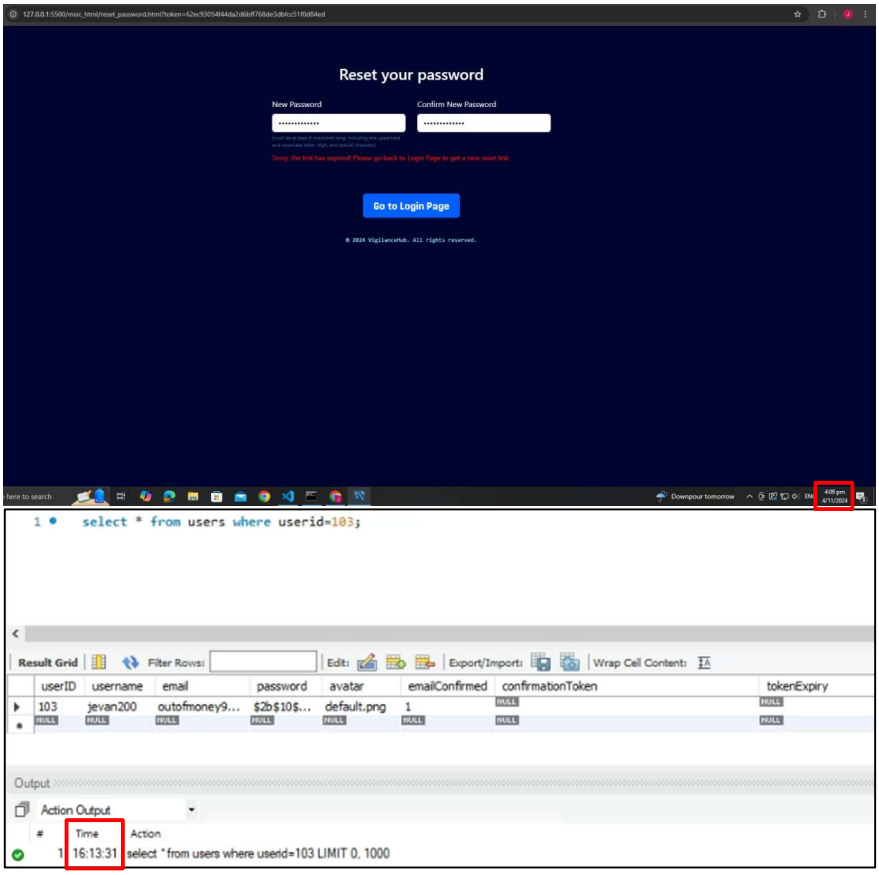
Upon doing this, jevan200 tries to access another web page in the auth_pages folder, **forum.html** in this instance by clicking on the 'Discussion' link on the navbar. This resulted in a 401 Unauthorized status being returned, redirecting to an interim page called **401.html**. Upon clicking the prompt that appears, jevan200 gets redirected back to **login.html** and must go through the standard login process again if he wishes to access the authenticated pages. A video on this process is demonstrated.

4.3. Exceeding the time limit given to reset password

** Refer to Section 3.8.6 for a more detailed explanation.*

When a user forgets their password, they can click the 'Forgot Password' option in **login.html** and they will be prompted to enter their email address. The user will then receive an email with a link to rest their password. User jevan200 forgot their password and enters their email to receive the link at 1555 hours, making the token expiry 10 minutes later at 1605 hours (do note that the scenarios in Figures 24a and 24b in Section 3.8.6 were used for this test). However, jevan200 failed to reset their password in time and only attempted to enter their new password at 1609 hours, four minutes past the expiry time. Therefore, **reset_password.html** gave an output stating that the reset password link has expired and that jevan200 had to redo the whole reset password process from the 'Forgot Password' option again. Figures 33a and 33b show the screenshots of the outputs as seen

from **reset_password.html** and the MySQL database (after the time limit was exceeded) respectively. A video on this process is demonstrated with a different example.



Figures 33a and 33b: Expired password reset and SQL query at 1613 hours

5. Implementation Details

This section documents the technical aspects of the VigilanceHub training platform. This includes the technologies, tools, and frameworks used to build the platform, along with the structure and integration between the frontend and backend. For the full source code and instructions on running VigilanceHub, do visit the GitHub repository through this link. The link is also found in the Appendices section at the end of this report.

5.1. Technologies Used

Table 4 describes the technologies used in the implementation of VigilanceHub, which are classified into programming languages, frameworks and databases.

Programming Languages	
HTML	Develop the main structure of the website which users would see on the frontend, such as web pages and the content.
CSS	Design the look and style of elements on the web pages such as buttons and containers. The CSS codes for the entire project are stored in **/public/css/vigilanceHub.css**.
JS	Controls the behaviours of elements, perform calculations on the website and fetches data from servers. Frontend JS codes for

	various functions are stored in the respective JS files in the folder **/public/js** (e.g. **register.js** and **forum.js**), while all backend JS codes are stored in **/src/app.js** from the main
Python	Used for writing a script to convert VigilanceHub into an executable file for an additional option for users to start the program.
JSON	Transmitting data within VigilanceHub from server to client and vice-versa, such as user data (userID, username, quiz scores etc.). Also stores the list of dependencies used in the project in a file called **package.json**.
Runtime Environments, Frameworks and Libraries	
Node.js	Provides the runtime environment for executing JS code on the backend in VigilanceHub.
Express.js	Backend framework which handles routing, middleware [15], and HTTP requests for VigilanceHub.
Nodemailer	A Node.js module that uses SMTP for sending emails from the backend.
CORS	A Node.js middleware that allows services running on different ports to communicate with each other. In VigilanceHub, the frontend and backend run on different ports and CORS is configured to allow communication between them.
Axios	HTTP client library that handles asynchronous requests and responses such as GET, POST, PUT and DELETE.
Express-session	Middleware library that runs between the request and response cycle in Express.js applications. In VigilanceHub, it handles session management, where server-side session data is stored and tracked for authentication in individual user sessions.
Databases	
MySQL	Stores all the user, posts and comments data.

Table 4: Technologies used in VigilanceHub

5.2. Tools and Dependencies

Various packages and libraries were installed in this Capstone to contribute to its full functionality. These dependencies and their version numbers are stored in the **package.json** file included in the main VigilanceHub directory, and they play crucial roles in both the backend and frontend functions. For example, **bcrypt** is used for secure password hashing, **express-session** handles session management to keep users authenticated across pages and **CORS** ensured smooth communication between the frontend and backend across different ports, to name a few.

6. Capstone Project Progress

6.1. Project Timeline

Table 5 shows the Capstone project timeline for VigilanceHub, from the beginning of the student's IWSP (January 2024) to the end of the Capstone project (December 2024).

Trimester	Month	Planned tasks
1	Jan	• Research on various large projects suitable for GRC • Propose initial idea to my academic supervisor
	Feb	• Finalise on Capstone Project idea and officially start project

	Mar	• Research on various features to include in the platform • Research on important GRC topics
	Apr	• Create question bank for quiz questions • Technical portion of the project • Integrating all components and troubleshooting
2	May	
	Jun	
	Jul	
	Aug	
	Sep	
3	Oct	• Capstone Project Report writing (Form E2) • Submission of Form E2 • Final Capstone Project presentation (26 Nov)
	Nov	

Table 5: Project timeline

For better clarity, a graphical form of the contents in Table 5 was created in the form of a Gantt chart.

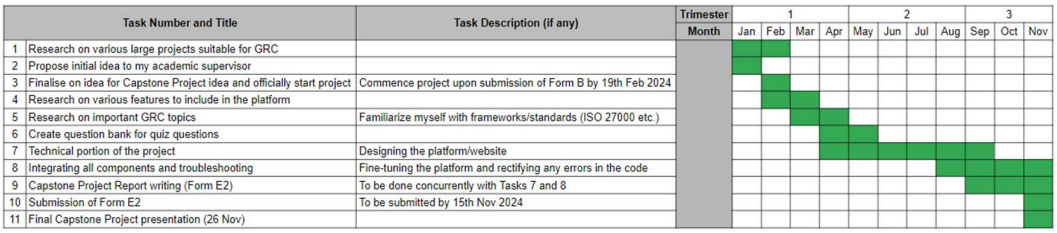


Figure 34: Project Gantt chart

7. Relevant Literature

As cybersecurity awareness has become increasingly important in the modern world, there are numerous articles explaining the benefits of inculcating such values and practices in the workplace and everyday life. Listed below are some articles that this Capstone project took inspiration from.

Firstly, Article [1] explains the need for security awareness training, a strategic approach utilised by IT and security professionals, with its purpose to educate employees and stakeholders on cybersecurity and data privacy, aiming to enhance awareness and mitigate cyber risks. The author also explained the difference between security awareness and security training and listed some benefits of awareness training, such as preventing financial loss and cultivating a cybersecurity mindset.

Next, Article [2] emphasizes the importance of a security awareness culture and how GRC plays a vital role in implementing such cultures within an organization. Statistics on the number of financial losses due to cybercrime were mentioned, and the authors provided six tips for employees to build the right GRC mindset, in addition to providing ten separate steps for an organization as a whole to build a strong security-aware culture.

In Article [3], the authors proposed a theoretical framework to improve cybersecurity awareness training programs. By categorizing programs based on costs and benefits, they highlight the importance of tailored approaches to mitigate cyber risks effectively. The study emphasizes the need for optimized resource allocation to develop impactful cybersecurity awareness training initiatives, offering valuable insights for enhancing overall cybersecurity readiness within organizations.

In Article [4], the authors used an example of the hospitality industry and highlighted its vulnerability to cyber threats due to employee awareness gaps and outdated training methods. A structured approach for an immersive cybersecurity awareness platform was proposed, leveraging gamification and industry insights to cultivate a culture of security.

In Article [5], the authors discuss the need for tailored cybersecurity awareness training and highlight the limitations of traditional rote methods. A training platform was developed in the form of an interactive video game designed to engage users while addressing organizational security objectives. The game proved to be a success for the training, showing promise as an effective addition to basic awareness training programs for general computer users.

Finally, the authors of Article [6] followed a similar approach to the previous article, where they brought up a possible conduct of cybersecurity awareness training in the form of games by reviewing various studies that explore the effectiveness of gaming technology to enhance cybersecurity awareness. The studies highlighted the rising importance of cybersecurity due to the increase in global internet use and cybercrimes, particularly in countries like Saudi Arabia. The main talking point in this article was about the potential of mobile gaming applications as an effective tool for creating security awareness. While most studies showed positive outcomes, the authors note the need for tailored and engaging gaming solutions to better address user needs.

8. Useful knowledge gained for the Capstone Project

8.1. Applicable Knowledge from the Degree Programme

Many of the modules taught in the degree programme, ICT(IS), have been of help for this Capstone project as they provided the backbone resources for the student to build this project on. The relevant modules and their relevancy to the Capstone project are listed in Table 6 below.

Module	Relevancy to project
ICT1002 - Programming Fundamentals	Python programming was first taught in this module, and the Python codes in this project involve developing a Python script for an alternate way to run the VigilanceHub application. Another Python script was coded to save the quiz questions and answers in a CSV file.
ICT1004 - Web Systems and Technologies	My program will be hosted on a local HTML web server, and I learnt about how to create HTML pages in this module, integrating them with CSS and JavaScript. In addition, my assignment in this module involved an E-commerce platform where items were stored in an SQL database and that was where I learned some fundamentals of it, which I am incorporating in this project as well.
ICT2203 - Network Security	Since my program is hosted on a local HTML web server, it uses the localhost IP address of 127.0.0.1. As I did not take ICT1010, Network Security was the first module in SIT where I learned about IP addresses and port numbers, and how they work together. My program runs on Port 5500 of the localhost, while Port 3000 is used for my Express.js server to listen to incoming HTTP requests. One of the training topics is also called Network Security,

	where users will learn about networking fundamentals such as IP/MAC addresses and commonly used network components to name a few.
ICT2205 - Applied Cryptography	In this module, I learned about various fundamentals in cryptography, such as salting and hashing, and some commonly used libraries. In this project, I have utilised two modules that Node.js offers to help enhance the security of VigilanceHub. One example is bcrypt, a password-hashing function, which I feel is rather simple to implement in my platform and I have since incorporated this function to hash the user's password whenever they register an account with VigilanceHub. Another example is the crypto module, where it was used to generate a 20-byte confirmation token used in the email confirmation feature.
ICT2206 - Web Security	As my program would be hosted locally, I am required to familiarise myself with common web application vulnerabilities so that I am aware of how to incorporate them in my training platform, such as proper session authentication where anyone can simply type in the URL of a page where users are required to sign in and access it if proper session management configurations were not incorporated. One of the training topics is also called Web Security, where users will learn about the URL, HTTP methods and port numbers to name a few.
ICT2208 - Security Governance, Risk Management and Compliance	I first learnt about GRC and various cybersecurity frameworks in this module, which got me interested in pursuing a career in cybersecurity specializing in the GRC sector and eventually choosing a GRC position for my IWSP. The platform that I have developed will incorporate security awareness training and one of the topics will briefly explain some various GRC frameworks and standards used in cybersecurity, such as ISO 27000 series. This Capstone project will also be beneficial to employees in my organization and the public as a security awareness training platform that strives to provide valuable learning resources to users.
ICT3201 - Operations Security and Incident Management	I learnt about the various plans that an organization will usually produce to aid with responding to security incidents. One of the topics in VigilanceHub will introduce users to the various documents and plans that would be useful for an organization, such as BCP and DRP.
ICT3203 - Secure Software Development	In this module, my project did involve session management in a website (as compared to ICT2206 where we only learned the basics of it), as part of an EC2 server that my group was required to utilise. I plan to incorporate session management into the VigilanceHub learning platform as well as this is one of the most important considerations in a proper functioning website.

	I also learned about the Node.js runtime environment and the Express.js framework in this module which I am using for building and running this platform.

Table 6: List of modules related to this project

8.2. Applicable Knowledge from beyond/outside the Degree Programme

In addition to the resources from the modules learnt in the degree programme, there are also several resources outside the degree programme that were of help for the completion of this Capstone Project.

8.2.1. Implementation of SQL Database

As user credentials need to be stored in a database, I felt that creating an SQL database is the most common and standard way to do so. Through online research, I have learnt how to incorporate an SQL database into my application, using MySQL Workbench [12, 13]. Although some of the Information Security modules in SIT involved SQL databases for assignments, my contributions to those assignments did not involve the SQL-related portions and/or connecting of the database, and therefore, I only learned some fundamentals of it back then. Connecting a backend database to store required data is essential for any application and I did more research to ensure that I am able to do so. For the end product, the MySQL database was successfully implemented and connects to the platform seamlessly.

Figure 35: Configuration in app.js to connect to the database

8.2.2. Sending emails via Nodemailer

The initial idea of implementing email functions within VigilanceHub was by using the popular Flask framework. However, I eventually decided to utilise Nodemailer [8] instead, which is a module of Node.js that allows users to send emails from the server. This was because the backend of VigilanceHub already runs via Node.js and Express.js, and including another backend (Flask) in a rather lightweight localhost application may overutilise unneeded resources. Therefore, Nodemailer was a more feasible option and would also allow me to explore more of the various features of Node.js.

Similar to Flask, Nodemailer uses SMTP to send emails. SMTP acts as the protocol for sending messages from one server to another, ensuring reliable delivery. While SMTP originally used Port 25 by default, Port 587 is preferred nowadays [9] as it caters for encrypted email transmissions which mitigates cyber-attacks such as email hijacking (often considered an MITM attack). With Nodemailer, email configuration is handled in JavaScript, where the SMTP settings, such as the host, port, and authentication credentials are defined. A transport object, aptly named 'transporter' is created using Nodemailer's *createTransport* method, which establishes a connection with the SMTP server. This is then followed by specifying the email details, such as the recipient, subject, and message body, using the *sendMail* method. Figures 36a and 36b show snippets of the code where the 'transporter' object is created, and the sending of the email using the *sendMail* method respectively.

```
const transporter = nodemailer.createTransport({
    host: 'smtp.gmail.com',
    port: 587,
    auth: {
        user: 'vigillancehub.help@gmail.com',
        pass: 'tbhb iwwv iwws gbbq'
    }
});
```

```
const message = `
    <p>You are registering an account with VigilanceHub. Please click the link below to confirm your registration within 10 minutes:</p>
    <a href="${confirmationLink}">Confirm your registration</a>
`;
transporter.sendMail({
    from: 'vigillancehub.help@gmail.com',
    to: email,
    subject: 'VigilanceHub Confirm Registration',
    html: message
}, (error, info) => {
    if (error) {
        console.error('Error sending email: ', error);
        res.status(500).json({ success: false, message: 'Failed to send confirmation email. Server error' });
        return;
    }
    console.log('Email sent successfully to ' + email);
    console.log('Details: ' + info.response + '\n');
});
```

Figures 36a and 36b: Creation of 'transporter' object and the code for the *sendMail* method

In Figure 36a, 'user' refers to the email address where emails are sent from, while 'pass' refers to the 16-character app password which allows less secure devices, such as the backend of VigilanceHub, to send emails.

8.2.3. Usage of CORS and Axios in the application

Cross-Origin Resource Sharing (CORS) [16] is utilised to enable secure interactions between the frontend and backend components of the platform. CORS is a mechanism that allows web applications running at one origin or domain to request resources from a different origin. In this setup, the frontend runs on port 5500, while the backend is served from port 3000. By configuring CORS, we specified that requests from the frontend are allowed to access resources on the backend server. This setup not only facilitates seamless data exchange between the client and server but also ensures that security policies are enforced to prevent unauthorized access. This configuration boosts the functionality of the platform, as it allows users to interact with various features and data without running into cross-origin issues, providing a smooth and integrated user experience.

The JavaScript library Axios [17] was also implemented and is used for making HTTP requests. Axios simplifies the process of sending and receiving data between the frontend and backend by providing a clean and intuitive API. In this project, Axios was used to manage user registration, authenticated login and logout to name a few. For instance, when users log in, Axios sends the user's credentials to the backend [21] and handles the response, including managing errors and updating the user interface based on the server's feedback. This use of Axios ensures that requests are handled efficiently and reliably, ensuring seamless communication flow between the client and server. Another Axios configuration that was necessary in facilitating secure user sessions is the *XMLHttpRequest.withCredentials* property [14]. This option ensures that session cookies, are included with requests, allowing the frontend to maintain authentication with the backend across different origins, essential for consistent and secure session handling. Some of the JS files that require user authentication like **login.js** and **edit_profile.js** have the *withCredentials* value set to **true** to ensure that user sessions are securely maintained, allowing authenticated requests to be validated by the server.

Figure 37: Snippet of code in login.js showing the withCredentials value set to true

9. Capstone Project limitations and challenges encountered

There were several challenges faced throughout the duration of the Capstone project, and these challenges resulted in several limitations which hindered VigilanceHub to reach its full potential as a robust platform for security awareness. This section documents the limitations and challenges faced by the student in developing VigilanceHub.

1. **Limitation: Potential scalability issues with the MySQL database**
   VigilanceHub utilises a MySQL database to store required data. While MySQL is capable of handling millions of rows of data, performance could become an issue as the platform grows, especially if the number of registered users hits into the thousands. With the potential large number of users, hundreds of posts and comments could be posted by these users over time in VigilanceHub's discussion forum, which could further exacerbate the performance of the database. Therefore, future considerations must be taken such as optimizing database queries or using more advanced database management techniques to ensure efficient data handling as the platform scales. In addition, the server running the database should also have a sufficient amount of RAM and storage space to ensure that it does not crash unexpectedly.

2. **Challenge: Issues implementing HTTPS across the website**
   Throughout the Capstone Project, VigilanceHub was developed entirely on a local server, and it was initially configured to run over HTTP, with links and resources statically pointing to local files within the project directory. HTTPS was considered only after completing the main functionality with the backend and UI features, which presented challenges due to HTTP-specific configurations and potential mixed content issues.

   I attempted to proceed with the HTTPS implementation [22] in the latter stages of this project by generating a self-signed SSL certificate, however, shifting the existing localhost setup to HTTPS required additional adjustments which were complex to resolve by then. From this experience, I learned that starting future projects with HTTPS from the beginning would better streamline its integration and ensure secure communication across all pages from the outset.

3. **Challenge: Conflicting issues while deploying the application to AWS**
   With the project being originally developed entirely on a local server, the idea of deploying the platform as a fully-fledged website did not surface during the initial stages of deployment. Therefore, VigilanceHub runs entirely on a localhost, or a single machine whereby copies of the application across different machines are unable to communicate with each other.

   I have attempted to deploy the application in its current form on the cloud-based UI

AWS Amplify. However, this required lots of troubleshooting and refactoring of the code from pointing to local paths within my device to using relative or absolute URLs that work in a live environment. Therefore, many, if not all the hardcoded local paths and links within the project, such as **http://127.0.0.1:5500/public/html/login.html**, need to be refactored to function in a cloud-based environment. Additionally, certain features that rely on session management or specific server configurations face compatibility issues when shifted to AWS. As a result, the application currently encounters functionality issues on AWS Amplify, and further adjustments are necessary to achieve full deployment.

The lack of successful deployment prevents the potential of better accessibility and real-world interaction. This has highlighted the importance of considering deployment requirements early in the development process to avoid extensive reconfiguration in the future.

The currently incomplete deployed project is available at: https://main.dsyejtbep7nfb.amplifyapp.com/.

10. Future improvements

This section documents several future improvements that have been planned for the Capstone project beyond the current timeline. In addition to the possibility of deploying this application on the public domain, a few other future improvements have been identified and are listed below.

10.1. Addition of new training topics

As VigilanceHub is planned as a platform to enhance users' knowledge in cybersecurity, it is important to keep the training content updated to ensure that users are constantly aware of the latest happenings in the cybersecurity landscape. Therefore, the current eight training topics must be updated whenever needed, with additions of new topics should the need arise. Some possible new topics that would be envisioned to stay relevant in the future could include:

- **AI Security**: Explore safe and secure uses of AI technologies and how to identify AI-generated cyber-attacks such as deepfakes.
- **Cloud Security**: Discuss the best practices of securing data and applications in cloud computing environments, such as AWS, Google Cloud and Microsoft Azure.
- **Mobile Security**: Address the growing risks associated with mobile devices, such as app vulnerabilities and mobile phishing attacks.
- **Organizational Cybersecurity Preparedness**: This would cater more to organizations for guidance on responding to cyber threats and implementing the proper prior measures, would introduce more frameworks such as COBIT 5 and HIPAA (the current **Topic 8 – Incident Response Fundamentals** is more of a touch-and-go topic for all types of users).

10.2. Phishing email feature

Another improvement that would enhance users' security awareness is a 'phishing email' feature. How this could work is that every interval of time, preferably 60 or 90 days, a phishing email will be sent to all registered users with a link included that users may or not click in. The contents of the phishing email may include one of the following:

- Urgent requests to update personal particulars
- Entering credentials to redeem free items sponsored by VigilanceHub
- User is requested to update their password to prevent their account getting compromised
- One-year free Netflix subscription if a user scores full marks for all quizzes (user again must enter credentials to 'connect' their account)

Each phishing email would include suspicious details for users to verify that the email is indeed not legitimate, which are explained in **Topic 2 – Understanding Cyber Threats** of VigilanceHub's training slides. The link included in the email will be harmless and simply leads to a HTML page with a message indicating that the user had opened a phishing email, reminding them to be more cautious in the future. The idea of a phishing email feature was inspired by my own IWSP company, where phishing emails are sent periodically to all employees with interesting details such as Valentine's Day offers and requesting to free up storage space in OneDrive. These emails are also harmless, and all employees will be notified a few days later about the nature of the email, reminding all employees, especially those who clicked the link, to be more vigilant in the future.

10.3. Implementing a proper frontend framework and deployment

The current frontend of VigilanceHub runs on the Go Live extension in Visual Studio Code, or Port 5500. While this setup is sufficient for basic functionality, implementing a well-established frontend framework like React.js could greatly enhance the platform's scalability, interactivity, and maintainability. This would allow VigilanceHub to manage complex user interactions and dynamic content more efficiently through a component-based architecture. This structure would improve the reusability of code and make future updates or feature additions more streamlined. Integrating a proper frontend framework like React.js would support future growth and provide a more seamless user experience, especially as the application expands to include more features and a larger number of users.

Secondly, implementing a framework like React would streamline cloud deployment on platforms like AWS Amplify, which I had initially attempted to deploy VigilanceHub on. React's build process creates files that are ready to work on the web as compared to just a single local device, thus eliminating the need for local paths. This simplifies deployment by ensuring the application can run smoothly on cloud platforms without needing special adjustments. With React, VigilanceHub could achieve faster, more reliable deployments, providing a scalable, maintainable structure that supports seamless updates and future growth as a fully deployed web application.

Finally, deploying the application with a proper frontend framework will also support HTTPS seamlessly, as the web pages in the deployed version of VigilanceHub would dynamically render content securely, unlike the static pages running on HTTP in the localhost version which was originally developed for this Capstone Project.

11. Conclusion

The VigilanceHub training platform successfully fulfils its goal of enhancing cybersecurity awareness through robust training materials, quizzes, and a discussion forum for users to interact with each other. This project addresses a critical need for accessible, foundational knowledge in cybersecurity, benefiting users of all levels.

Throughout the past nine months in Wizlynx Pte Ltd after this Capstone Project commenced in February, other employees in the company have tried out the website and gave positive feedback highlighting its usability and relevance. In the future, VigilanceHub will continue to improve and cater to a wider target group.

References

[1] K. Yasar, "What is Security Awareness Training?" [Online]. Available: https://www.techtarget.com/searchsecurity/definition/security-awareness-training.

[2] StandardFusion, "Nurturing an Unbreachable Business: The Importance of a Security-Aware Culture," 10 July 2023 [Online]. Available: https://www.standardfusion.com/blog/security-awareness-culture-and-grc-in-businesses/.

[3] Z. Zhang, W. He, W. Li, M. Abdous, "Cybersecurity awareness training programs: a cost–benefit analysis framework," 27 January 2021 [Online]. Available: https://www.emerald.com/insight/content/doi/10.1108/IMDS-08-2020-0462/full/html.

[4] J. Holdsworth, E. Apeh, "An Effective Immersive Cyber Security Awareness Learning Platform for Businesses in the Hospitality Sector," 2 October 2017 [Online]. Available: https://ieeexplore.ieee.org/abstract/document/8054838.

[5] B.D. Cone, C.E. Irvine, M.F. Thompson, T.D. Nguyen, "A video game for cyber security training and awareness," 28 November 2006 [Online]. Available: https://www.sciencedirect.com/science/article/abs/pii/S0167404806001556.

[6] F. Alotaibi, S. Furnell, I. Stengel, M. Papadaki, "A Review of Using Gaming Technology for Cyber-Security Awareness," 2 June 2016 [Online]. Available: https://infonomics-society.org/wp-content/uploads/ijisr/published-papers/volume-6-2016/A-Review-of-Using-Gaming-Technology-for-Cyber-Security-Awareness.pdf.

[7] W3Schools, "JavaScript Date setMinutes()," [Online]. Available: https://www.w3schools.com/jsref/jsref_setminutes.asp.

[8] Nodemailer [Online]. Available: https://nodemailer.com/.

[9] Mailgun, "Which SMTP Port to Use? Understanding ports 25, 465, & 587," 26 July 2024 [Online]. Available: https://www.mailgun.com/blog/email/which-smtp-port-understanding-ports-25-465-587/.

[10] R. Hendel, "How session and cookies works ?" 4 March 2023 [Online]. Available: https://medium.com/@hendelRamzy/how-session-and-cookies-works-640fb3f349d1.

[11] Descope, "What Is Session Management & Tips to Do It Securely," 30 March 2024 [Online]. Available: https://www.descope.com/learn/post/session-management.

[12] Express.js, "Database integration," [Online]. Available: https://expressjs.com/en/guide/database-integration.html.

[13] A. Darji, "Building a MySQL Database Connection with Node.js and Express," 27 April 2024 [Online]. Available: https://anujdarji100737.medium.com/building-a-mysql-database-connection-with-node-js-and-express-b6b333f6e713.

[14] MDN Web Docs, “XMLHttpRequest: withCredentials property,” 26 July 2024 [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/XMLHttpRequest/withCredentials.

[15] Express.js, “Writing middleware for use in Express apps,” [Online]. Available: https://expressjs.com/en/guide/writing-middleware.html.

[16] MDN Web Docs, “Cross-Origin Resource Sharing (CORS),” 2 October 2024 [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/HTTP/CORS.

[17] F. Kelhini, “How to make HTTP requests with Axios,” 29 November 2023 [Online]. Available: https://blog.logrocket.com/how-to-make-http-requests-like-a-pro-with-axios/.

[18] S. Ulili, C. Postal, “How To Read and Write CSV Files in Node.js Using Node-CSV,” 22 April 2022 [Online]. Available: https://www.digitalocean.com/community/tutorials/how-to-read-and-write-csv-files-in-node-js-using-node-csv.

[19] Bertrand, “Delete account design: inspiration tips and best practices,” [Online]. Available: https://nicelydone.club/blog/delete-account-examples-inspiration.

[20] J. Coutinho, “How to Implement a Forgot Password Flow? Complete Guide,” 13 July 2024 [Online]. Available: https://supertokens.com/blog/implementing-a-forgot-password-flow.

[21] C. Innocent, “How to use Axios POST requests,” 1 July 2024 [Online]. Available: https://blog.logrocket.com/how-to-use-axios-post-requests/.

[22] Steve Griffith - Prof3ssorSt3v3, “Add HTTPS support to VSCode Live Server,” 23 July 2019 [Online]. Available: https://www.youtube.com/watch?v=v4jgr0ppw8Q&t=239s.

Appendices

GitHub Repository

https://github.com/jevancjk/VigilanceHub

Data Directory

Abbreviation	Meaning
AAA	Authentication, Authorization, Accounting
AI	Artificial Intelligence
API	Application Programming Interface
AWS	Amazon Web Services
CIA	Confidentiality, Integrity, Availability
COBIT	Control Objectives for Information and Related Technologies
CRUD	Create, Read, Update, Delete
CSS	Cascading Style Sheets
CSV	Comma-Separated Value
EC2	Elastic Compute Cloud
GRC	Governance, Risk Management & Compliance
HIPAA	Health Insurance Portability and Accountability Act

HTML	Hypertext Markup Language
HTTP / HTTPS	Hypertext Transfer Protocol / Hypertext Transfer Protocol Secure
IP	Internet Protocol
ISO	International Organization for Standardization
IWSP	Integrated Work Study Programme
JS	JavaScript
JSON	JavaScript Object Notation
MAC	Media Access Control
MITM	Man-in-the-Middle
NIST	National Institute of Standards and Technology
OSI	Open Systems Interconnection
PDF	Portable Document Format
RAM	Random-access Memory
SIT	Singapore Institute of Technology
SMTP	Simple Mail Transfer Protocol
SQL	Structured Query Language
SSL	Secure Sockets Layer
UI	User Interface
URL	Uniform Resource Locator

END OF DOCUMENT